

QuantLab

Technical Documentation

Mathematical Foundations, Algorithms
& Implementation Guide

For Quantitative Analysts, Portfolio Managers & Developers

April 03, 2026

Table of Contents

1. Architecture Overview

2. Market Data & Performance Metrics

3. Valuation Models

- 3.1 CAPM
- 3.2 WACC
- 3.3 DCF
- 3.4 Fama-French 3-Factor
- 3.5 APT

4. Portfolio Optimization

- 4.1 Maximum Sharpe Ratio
- 4.2 Minimum Variance
- 4.3 Risk Parity
- 4.4 Minimum CVaR
- 4.5 Maximum Diversification
- 4.6 Kelly Criterion
- 4.7 Black-Litterman
- 4.8 Hierarchical Risk Parity
- 4.9 Equal Weight

5. Bubble Detection

- 5.1 Metcalfe's Law
- 5.2 GPH Long-Memory Estimator
- 5.3 Kurtosis & Ljung-Box
- 5.4 Composite Bubble Score

6. Monte Carlo Simulation

- 6.1 Geometric Brownian Motion
- 6.2 Behavioral Agent Model
- 6.3 Risk Metrics

7. Technical Analysis

8. Options Pricing

- 8.1 Black-Scholes
- 8.2 Greeks
- 8.3 Implied Volatility Surface
- 8.4 Payoff Diagrams

9. Risk & Geopolitics Dashboard

- 9.1 Cross-Asset Signals
- 9.2 Yield Curve Analysis
- 9.3 Composite Risk Score

10. Machine Learning — Predictive Models

- 10.1 Linear Regression
- 10.2 Random Forest
- 10.3 Gradient Boosting

11. ML Clustering & Regime Detection

- 11.1 K-Means Clustering
- 11.2 Gaussian Mixture
- 11.3 PCA Reduction
- 11.4 Anomaly Detection

12. Sentiment Analysis

- 12.1 News Fetching
- 12.2 Keyword Scoring
- 12.3 Aggregate Trends

13. Export System

- 13.1 PDF Report
- 13.2 Slide Deck
- 13.3 Excel Workbook

14. Configuration & Advanced Settings

15. Multi-Source Data Layer

- 15.1 DataProviderBase
- 15.2 YFinanceProvider
- 15.3 AlphaVantageProvider
- 15.4 FREDProvider & FinnhubProvider
- 15.5 DataSourceOrchestrator

16. Backtesting Engine

- 16.1 Walk-Forward Framework
- 16.2 Strategy Implementations
- 16.3 Performance Metrics

17. Fundamental Data Panel

- 17.1 Key Ratios
- 17.2 Financial Statements

18. Fixed Income & Macro Analytics

- 18.1 Bond Pricing
- 18.2 Duration & Convexity
- 18.3 Yield Curve Analysis

19. Multi-Factor Alpha Model

- 19.1 Factor Construction
- 19.2 Factor Timing
- 19.3 Alpha Attribution

20. Options Strategy Builder

- 20.1 Strategy Templates
- 20.2 Multi-Leg Builder
- 20.3 Options Screener

21. Portfolio Risk Suite

- 21.1 Historical Stress Tests
- 21.2 Custom Scenarios
- 21.3 Factor VaR & Correlation Breakdown

22. Pairs Trading & Statistical Arbitrage

- 22.1 Cointegration Testing
- 22.2 Spread & Z-Score
- 22.3 Half-Life of Mean Reversion
- 22.4 Signal Generation & Backtest

23. Sector Rotation & Relative Strength

- 23.1 Momentum Rankings
- 23.2 Regime Detection
- 23.3 Sector Correlation

24. Earnings & Event Analysis

- 24.1 Post-Earnings Drift
- 24.2 Event Volatility
- 24.3 Seasonality

25. Tail Risk & Drawdown Analysis

- 25.1 EVT / GPD Fit
- 25.2 Drawdown Episodes
- 25.3 Distribution Analysis
- 25.4 Tail Dependence

26. Currency & Cross-Asset Correlation

- 26.1 Dynamic Correlations
- 26.2 Regime Detection
- 26.3 Currency Strength

27. ESG & Alternative Data Scoring

- 27.1 ESG Scores
- 27.2 Peer Ranking
- 27.3 ESG-Return Analysis

28. Enhanced Pairs Trading Backtest

- 28.1 Kalman Filter Hedge Ratio
- 28.2 Walk-Forward Optimization
- 28.3 Regime-Adaptive Thresholds

29. Macro Regime Detector

- 29.1 Indicator Signals
- 29.2 Composite Regime
- 29.3 Asset Performance by Regime
- 29.4 Transition Matrix

30. Cryptocurrency On-Chain Metrics

- 30.1 NVT Ratio
- 30.2 MVRV Proxy
- 30.3 Fear & Greed Index

31. Insider Trading Tracker

- 31.1 Insider Sentiment Score
- 31.2 Cluster Detection
- 31.3 Forward Return Analysis

32. Watchlist & Alerts

- 32.1 Watchlist Management

32.2 Alert Engine

32.3 Performance Heatmap

33. ML-Powered Top 10 Assets

33.1 Feature Engineering

33.2 Ensemble Scoring

33.3 Asset Explanations

33.4 Model Diagnostics

34. News Sentiment Analyzer

34.1 TF-IDF + Naive Bayes

34.2 Divergence Detection

34.3 Sentiment-Price Correlation

35. Smart Portfolio Constructor

35.1 Ledoit-Wolf Covariance

35.2 Black-Litterman Framework

35.3 ML-Derived Views

35.4 Regime Monte Carlo

36. Risk Decomposition Engine

36.1 Factor Regression

36.2 Return & Risk Attribution

36.3 Style Box Classification

37. ML Price Forecaster

37.1 Rolling Regression

37.2 Gradient Boosting

37.3 ARIMA + EWMA Ensemble

38. Earnings Surprise Predictor

38.1 Pre-Earnings Features

38.2 RandomForest Classifier

38.3 Walk-Forward Validation

A. Appendix: Supported Asset Classes

B. Appendix: Dependencies

C. Appendix: Data Source Configuration

D. Appendix: ML Model Specifications

1. Architecture Overview

QuantLab is an institutional-grade multi-asset quantitative research platform built with Streamlit, covering the complete investment research workflow across 36 analytical tabs. The monolithic single-file architecture (app.py) enables zero-config deployment on Streamlit Cloud while housing all analytical modules, UI rendering, data fetching, and export logic in a single cohesive namespace.

Purpose & Role

The architecture layer defines the application's technology stack, data flow patterns, and module organization. It ensures that each analytical component can access shared data (via Streamlit session state) while maintaining clean separation of concerns through class-based encapsulation.

Technology Stack

Layer	Technology
Frontend / UI	Streamlit 1.56, Plotly
Data Ingestion	yfinance, Alpha Vantage, FRED, Finnhub
Numerical	NumPy, SciPy, StatsModels
Machine Learning	scikit-learn (RF, GB, LR, K-Means, GMM, PCA, Isolation Forest, ComplementNB, TF-IDF)
Technical Indicators	ta library (RSI, MACD, Bollinger Bands, SMA, EMA)
Visualization	Plotly (interactive), Matplotlib/Seaborn (exports)
Export	fpdf2 (PDF/slides), xlsxwriter (Excel)
Testing	pytest, streamlit.testing.v1 (590+ tests)
CI/CD	GitHub Actions (Python 3.11 & 3.12)

Data Flow

User inputs tickers and parameters in the sidebar. The DataSourceOrchestrator fetches prices via a cascading fallback chain (yfinance -> Alpha Vantage -> Finnhub -> cache). Prices are stored in session state as a shared DataFrame. Each tab's render function reads from session state, runs its computations, and writes results back. The export system reads all accumulated results to generate PDF, slides, and Excel outputs.

Cross-Module Interactions

Bubble detection scores feed into portfolio optimization (bubble-aware penalty). Valuation metrics inform the Monte Carlo behavioral agent model. The backtesting engine consumes portfolio weights from any optimization strategy. Factor model exposures are reused by the risk decomposition engine. ML predictions feed the smart portfolio constructor's view generation.

Error Handling

QuantLab uses a typed exception hierarchy: QuantLabError (base), DataFetchError, ValidationError, CalculationError, and ExportError. The handle_error decorator wraps all

computation functions, providing user-friendly error messages with recovery hints. An 8-step progress bar tracks analysis progress, and individual module failures produce warnings without crashing the pipeline.

Session State Keys

QuantLab uses Streamlit session state as a shared data bus. Key state variables include:

Key	Type	Description
prices_data	pd.DataFrame	OHLCV price matrix for all selected tickers
returns_data	pd.DataFrame	Daily log returns matrix
bubble_scores	dict	Per-ticker bubble detection scores (0-1)
portfolio_weights	dict	Current optimal portfolio weights
ml_models	dict	Trained ML model objects and metrics
risk_data	dict	Composite risk score and component data
backtest_results	dict	Equity curve, metrics, trade log
factor_scores	pd.DataFrame	Cross-sectional factor exposures
regime_label	str	Current macro regime classification
watchlists	dict	Named watchlists with alert configs
orchestrator	DataSourceOrchestrator	Configured data provider cascade

Rendering Pipeline

Each tab follows a consistent rendering pattern: (1) read shared data from session state, (2) compute tab-specific analytics, (3) render Plotly charts and Streamlit widgets, (4) write results back to session state for downstream consumption. Error handling wraps each computation in a try/except block with graceful degradation.

Theme System

QuantLab supports light and dark themes via `inject_custom_css()`. The theme propagates to all Plotly charts via a centralized `_get_plotly_theme()` function that returns consistent colors, fonts, and layout settings. Tables use `render_styled_table()` with theme-aware alternating row colors.

2. Market Data & Performance Metrics

This foundational module retrieves and processes market data for all downstream analytics. It provides standardized access to OHLCV price data, dividend-adjusted returns, and key performance metrics across all supported asset classes.

The Market Dashboard tab provides a normalized price history chart and key performance metrics for all selected tickers. It serves as the entry point for analysis, displaying annual return, annualized volatility, Sharpe ratio, and maximum drawdown.

Purpose & Role

This module computes and displays the core descriptive statistics that underpin all downstream analyses. By normalizing prices to a common base (100), it enables visual comparison across assets with different price scales.

Core Metrics

```
Annual Return = (P_end / P_start)252/n - 1  
Volatility = std(daily_returns) * sqrt(252)  
Sharpe Ratio = (R_p - R_f) / sigma_p  
Max Drawdown = max(peak - trough) / peak
```

Cross-Module Interactions

Price data from this module feeds into every other tab. Returns serve as input to portfolio optimization, bubble detection, ML models, and risk analytics. The `fetch_market_data()` function is the primary data gateway.

Implementation

```
def fetch_market_data(tickers, start_date, end_date):  
    """Fetch OHLCV via DataSourceOrchestrator."""  
    orch = _get_orchestrator()  
    return orch.fetch_prices(tickers, period)
```

Normalization Method

Price normalization divides each ticker's price series by its initial value and multiplies by 100, creating a common base for visual comparison. This preserves return patterns while eliminating scale differences between assets priced at different levels.

```
Normalized_t = (P_t / P_0) * 100
```

Data Quality Handling

Missing data is handled via forward-fill followed by backward-fill. Tickers with more than 30% missing data are excluded with a warning. Weekend and holiday gaps are handled automatically by `DatetimeIndex` alignment. Volume data of zero is replaced with NaN to avoid distorting volume-based indicators.

2.4 Performance Metrics Calculation

The platform computes a comprehensive set of performance metrics for every analyzed asset:

Metric	Formula	Interpretation
CAGR	$(P_{\text{end}}/P_{\text{start}})^{1/n} - 1$	Annualized compound growth rate
Annualized Volatility	$\sigma_{\text{daily}} * \sqrt{252}$	Annualized standard deviation of returns
Sharpe Ratio	$(R_p - R_f) / \sigma_p$	Risk-adjusted excess return per unit volatility
Sortino Ratio	$(R_p - R_f) / \sigma_{\text{downside}}$	Penalizes only downside volatility
Max Drawdown	$\max(1 - P_t/P_{\text{peak}})$	Worst peak-to-trough decline
Calmar Ratio	CAGR / Max Drawdown	Return per unit of maximum drawdown
Beta	$\text{Cov}(R_p, R_m) / \text{Var}(R_m)$	Sensitivity to market returns
Alpha (Jensen)	$R_p - [R_f + \beta(R_m - R_f)]$	Excess return after adjusting for beta
Information Ratio	$(R_p - R_b) / \text{TE}$	Active return per unit of tracking error
Skewness	$E[(R - \mu)^3] / \sigma^3$	Asymmetry of return distribution
Kurtosis	$E[(R - \mu)^4] / \sigma^4 - 3$	Tail heaviness (excess kurtosis)

2.5 Data Retrieval Architecture

Market data is retrieved via the yfinance library with configurable date ranges. The default lookback period is 2 years for price data and 5 years for fundamental data. All timestamps are normalized to UTC and resampled to the requested frequency (daily, weekly, monthly). Missing data is forward-filled for up to 5 business days; gaps exceeding this threshold are flagged as data quality issues.

```
class MarketDataFetcher:
    def fetch_price_data(self, tickers: list, period: str = "2y") -> pd.DataFrame
    def compute_returns(self, prices: pd.DataFrame, method: str = "log") -> pd.DataFrame
    def compute_metrics(self, returns: pd.DataFrame) -> dict
    def get_benchmark_data(self, benchmark: str = "SPY") -> pd.DataFrame
    def validate_data_quality(self, df: pd.DataFrame) -> dict # {missing_pct, gaps, staleness}
```

2.6 Return Computation Methods

The platform supports both arithmetic and logarithmic returns. Log returns are preferred for multi-period aggregation (time-additivity), while arithmetic returns are used for portfolio-level calculations (cross-sectional additivity):

$$R_{\text{log}} = \ln(P_t / P_{t-1})$$

$$R_{\text{arith}} = (P_t - P_{t-1}) / P_{t-1}$$

Dividend-adjusted returns incorporate total return by using the Adj Close series from Yahoo Finance, which back-adjusts historical prices for splits and dividends.

3. Valuation Models

Five valuation models are computed per ticker using live fundamental data from yfinance. The `EnhancedValuationMetrics` class provides static methods for each model, enabling standalone or combined use.

3.1 CAPM

The Capital Asset Pricing Model estimates expected return based on systematic risk exposure. Beta is computed via OLS regression of asset returns against benchmark returns (S&P 500).

$$E(R) = R_f + \text{beta} * (E(R_m) - R_f)$$

$$\text{beta} = \text{Cov}(R_i, R_m) / \text{Var}(R_m)$$

The equity risk premium (ERP) defaults to 5.5%. The risk-free rate is sourced from the 10-year Treasury yield via yfinance (^TNX).

```
EnhancedValuationMetrics.calculate_beta(ticker_prices, benchmark_prices)
EnhancedValuationMetrics.calculate_capm_return(rf_rate, beta)
```

3.2 WACC

Weighted Average Cost of Capital blends the cost of equity (CAPM-derived) with the after-tax cost of debt, weighted by capital structure.

$$\text{WACC} = (E/(E+D)) * R_e + (D/(E+D)) * R_d * (1 - t)$$

Where E = market cap, D = total debt, R_e = CAPM return, R_d = interest expense / total debt, t = effective tax rate. If debt data is unavailable, WACC defaults to cost of equity.

```
EnhancedValuationMetrics.calculate_wacc(ticker, rf_rate, beta)
```

3.3 DCF

Discounted Cash Flow valuation projects free cash flows and discounts them at WACC. A terminal value is computed using a perpetuity growth model with a growth-rate guard (g must be less than WACC).

$$V = \sum_{t=1}^n \text{FCF}_t / (1 + \text{WACC})^t + \text{TV} / (1 + \text{WACC})^n$$

$$\text{TV} = \text{FCF}_n * (1 + g) / (\text{WACC} - g)$$

Growth rate is capped at WACC - 1% to ensure convergence. Free cash flow is sourced from yfinance cash flow statements. Projection horizon defaults to 5 years with a 3% terminal growth rate.

```
EnhancedValuationMetrics.calculate_dcf_value(ticker, rf_rate, beta)
```

3.4 Fama-French 3-Factor

Extends CAPM by adding size (SMB) and value (HML) factor exposures. Factor returns are approximated from yfinance using IWM-SPY for SMB and IWD-IWF for HML.

$$E(R) = R_f + \text{beta}_m * (R_m - R_f) + \text{beta}_s * \text{SMB} + \text{beta}_h * \text{HML}$$

```
EnhancedValuationMetrics.calculate_fama_french_return(ticker, prices, rf_rate,
beta)
```

3.5 APT

Arbitrage Pricing Theory estimates expected return using multiple macro factors. QuantLab uses a simplified version with market, size, and momentum factors.

$$E(R) = R_f + \text{Sum}(\text{beta}_k * \text{lambda}_k)$$

```
EnhancedValuationMetrics.calculate_apr_return(ticker, prices, rf_rate)
```

Cross-Module Interactions

Valuation results feed into the bubble detection module -- the DCF-implied value provides a fundamental anchor for the Behavioral Agent Monte Carlo simulator. CAPM beta is reused by the portfolio risk suite for stress test parametric fallbacks. WACC serves as the discount rate in terminal value calculations across modules.

Bubble-Burst Impact Analysis

An additional method estimates the potential price decline if a detected bubble were to burst:

$$\text{Impact} = \text{Current_Price} * (1 - \text{beta} * \text{bubble_score} * \text{correction_factor})$$

Where `correction_factor` is calibrated from historical bubble episodes (default: 0.30 for moderate correction, 0.50 for severe).

```
EnhancedValuationMetrics.calculate_bubble_burst_impact(ticker, prices,
bubble_score, beta)
```

Data Requirements

Valuation models require at minimum 252 trading days of price history for robust beta estimation. Fundamental data (revenue, EBITDA, free cash flow, debt, equity) is sourced from yfinance quarterly and annual statements. If fundamental data is unavailable (e.g., for crypto or forex), valuation tabs display a warning and skip those models.

4. Portfolio Optimization

The `EnhancedPortfolioOptimizer` class implements nine optimization strategies. All strategies accept a `bubble_aware` flag that penalizes weights toward assets with high bubble scores.

4.1 Maximum Sharpe Ratio

Maximizes risk-adjusted return by solving a constrained optimization problem via `scipy.optimize.minimize` (SLSQP).

$$\begin{aligned} \max \quad & w'\mu - (\lambda/2) * w'\Sigma*w \\ \text{subject to:} \quad & \text{Sum}(w_i) = 1, w_i \geq 0 \end{aligned}$$

```
EnhancedPortfolioOptimizer.maximum_sharpe(bubble_aware=False, penalty_factor=0.5)
```

4.2 Minimum Variance

Minimizes portfolio variance without regard to expected returns.

$$\min w'\Sigma*w \text{ subject to: } \text{Sum}(w_i) = 1, w_i \geq 0$$

```
EnhancedPortfolioOptimizer.minimum_variance(bubble_aware, penalty_factor)
```

4.3 Risk Parity

Equalizes the marginal risk contribution of each asset so that each contributes equally to total portfolio variance.

$$w_i * (\Sigma*w)_i = (1/n) * w'\Sigma*w \text{ for all } i$$

Implemented via optimization: minimize the sum of squared deviations of risk contributions from the target (1/n of total risk).

```
EnhancedPortfolioOptimizer.risk_parity(bubble_aware, penalty_factor)
```

4.4 Minimum CVaR

Minimizes Conditional Value at Risk (Expected Shortfall) at the 5% level, targeting worst-case tail losses.

$$\text{CVaR}_\alpha = -E[R \mid R \leq \text{VaR}_\alpha]$$

```
EnhancedPortfolioOptimizer.minimum_cvar(alpha=0.05)
```

4.5 Maximum Diversification

Maximizes the diversification ratio: the ratio of the weighted sum of individual volatilities to portfolio volatility.

$$\text{DR} = (\text{Sum } w_i * \sigma_i) / \sqrt{w'\Sigma*w}$$

4.6 Kelly Criterion

Computes the growth-optimal portfolio weights with an optional leverage limit.

$$w^* = \text{Sigma}^{-1} * \mu / \text{lambda}$$

```
EnhancedPortfolioOptimizer.kelly_criterion(leverage_limit=1.0)
```

4.7 Black-Litterman

Combines market equilibrium returns with investor views using Bayesian updating.

$$\mu_{BL} = [(\tau * \text{Sigma})^{-1} + P' * \Omega^{-1} * P]^{-1} * [(\tau * \text{Sigma})^{-1} * \text{Pi} + P' * \Omega^{-1} * Q]$$

Where Pi = market-implied equilibrium returns, P = view matrix, Q = view returns, Omega = view uncertainty, tau = scalar (default 0.05).

```
EnhancedPortfolioOptimizer.black_litterman(market_cap_weights, views,
view_confidence)
```

4.8 Hierarchical Risk Parity (HRP)

Uses hierarchical clustering on the correlation matrix to determine portfolio weights. The algorithm: (1) compute distance matrix from correlation, (2) apply single-linkage clustering, (3) quasi-diagonalize the covariance matrix, (4) recursively bisect to allocate weights inversely proportional to cluster variance.

```
EnhancedPortfolioOptimizer.hierarchical_risk_parity()
```

4.9 Equal Weight

Naive 1/N allocation. Serves as the benchmark for comparing optimization strategies.

$$w_i = 1/n \text{ for all } i$$

Bubble-Aware Penalty

When enabled, each asset's weight is penalized by its bubble score: $\text{penalty} = \text{penalty_factor} * \text{Sum}(w_i * \text{bubble_score}_i)$. This term is added to the objective function, discouraging allocation to assets flagged as overvalued by the bubble detection module.

Efficient Frontier

The `efficient_frontier()` method generates 100 portfolios along the mean-variance frontier by varying the target return from minimum-variance to maximum-return. Each point is plotted as (volatility, return) with the Sharpe-optimal portfolio highlighted.

Portfolio Metrics

After optimization, comprehensive portfolio metrics are computed:

Metric	Formula
Expected Return	$w' * \mu$ (annualized)
Portfolio Volatility	$\text{sqrt}(w' * \text{Sigma} * w) * \text{sqrt}(252)$
Sharpe Ratio	$(\text{Return} - R_f) / \text{Volatility}$
Sortino Ratio	$(\text{Return} - R_f) / \text{Downside Deviation}$
Maximum Drawdown	max peak-to-trough decline

Metric	Formula
Calmar Ratio	CAGR / Max Drawdown
VaR (95%)	5th percentile of return distribution
CVaR (95%)	Mean of returns below VaR
Diversification Ratio	Weighted avg vol / portfolio vol
Concentration (HHI)	Sum of squared weights

```
EnhancedPortfolioOptimizer.calculate_portfolio_metrics(weights)
```

Configuration

- Bubble-Aware toggle: `st.session_state['bubble_aware']`
- Penalty factor: `st.session_state['bubble_penalty']` (0.0-2.0, default 0.5)
- Benchmark: `st.session_state['benchmark']` (S&P 500 / NASDAQ / None)
- Risk-free rate: `st.session_state['rf_rate']` (default: 10Y Treasury)

5. Bubble Detection

The BubbleDetector class combines four independent indicators into a composite bubble score (0-1). Each indicator targets a different market pathology.

5.1 Metcalfe's Law

The MetcalfeLawAdvanced class estimates fundamental network value using trading volume as a proxy for network activity. The Market-to-Metcalfe Value (MMV) ratio compares market price to the model-implied value.

$$NV = k * v^{\alpha} \text{ (generalized Metcalfe, } \alpha \approx 1.5)$$

$$MMV = \text{Market Price} / \text{Network Value}$$

MMV > 1 suggests overvaluation; MMV > 2 flags bubble territory. Regime thresholds: Normal (MMV < 1.5), Caution (1.5-2.0), Bubble (> 2.0).

5.2 GPH Long-Memory Estimator

The Geweke-Porter-Hudak estimator detects long-memory (fractional integration) in return series. A parameter $d > 0.5$ indicates non-stationary, bubble-like persistence.

$$d_{GPH} = -0.5 * \text{slope of } \log(\text{periodogram}) \text{ vs } \log(\text{frequency})$$

Standard error is computed as $\pi / \sqrt{24 * m}$ where m is the bandwidth parameter (default: $n^{0.5}$).

```
LongMemoryEstimators.gph_estimator(returns, bandwidth=None)
```

5.3 Kurtosis & Ljung-Box

Excess kurtosis measures fat tails in the return distribution. Values above 5 indicate elevated speculative risk. The Ljung-Box test on squared returns detects volatility clustering (ARCH effects), a hallmark of speculative markets.

5.4 Composite Bubble Score

The four indicators are normalized to [0, 1] and combined via equal weighting:

$$\text{Bubble Score} = 0.25 * \text{MMV_score} + 0.25 * \text{GPH_score} + 0.25 * \text{Kurtosis_score} + 0.25 * \text{LB_score}$$

Regime labels: Normal (0-0.33), Caution (0.33-0.66), High Risk (0.66-1.0). Scores feed into bubble-aware portfolio optimization.

Score Normalization Details

Raw indicator values are mapped to [0, 1] via sigmoid-like transformations calibrated to historical distributions:

- MMV score: $\text{sigmoid}((\text{MMV} - 1.0) / 0.5)$ -- centered at MMV = 1.0
- GPH score: $\text{sigmoid}((d - 0.5) / 0.2)$ -- centered at $d = 0.5$
- Kurtosis score: $\min(1, \max(0, (\text{kurtosis} - 3) / 10))$

- Ljung-Box score: $1 - p_value$ ($p < 0.05$ implies clustering)

Cross-Module Propagation

Bubble scores propagate to: (1) Portfolio Optimization -- penalty term in objective function, (2) Backtesting Engine -- bubble-tilted weights at rebalance dates, (3) Monte Carlo -- Behavioral Agent model calibrates speculator fraction from bubble score, (4) Valuation -- bubble-burst impact estimation, (5) Market Dashboard -- regime labels displayed per ticker.

6. Monte Carlo Simulation

6.1 Geometric Brownian Motion

Standard log-normal price paths calibrated to historical drift and volatility.

$$dS = \mu * S * dt + \sigma * S * dW$$

$$S(t+dt) = S(t) * \exp((\mu - \sigma^2/2) * dt + \sigma * \sqrt{dt} * Z)$$

Where $Z \sim N(0,1)$. Parameters μ and σ are estimated from historical log-returns. Default: 500 simulations, 90-day horizon.

6.2 Behavioral Agent Model

The BehavioralAgentSimulator implements a heterogeneous agent model mixing fundamentalist (mean-reversion) and speculator (momentum) demand. Calibrated to historical beta, autocorrelation, and sector dynamics.

```
class BehavioralAgentSimulator:
    def __init__(self, ticker, price_series)
    def run(self, n_days=252, n_sims=1000)
```

6.3 Risk Metrics

Outputs from both engines: median price path, Value at Risk (VaR), Conditional VaR (CVaR), 90% confidence band, and regime labels (Bubble / Fair / Undervalued) based on simulation endpoints relative to current price.

$$VaR_{\alpha} = -\text{quantile}(\text{returns}, \alpha)$$

$$CVaR_{\alpha} = -E[R \mid R \leq -VaR_{\alpha}]$$

Behavioral Agent Model Details

The heterogeneous agent model simulates a market with two agent types:

Agent Type	Demand Function	Behavior
Fundamentalist	$D_f = \phi * (P_{\text{fair}} - P_{\text{current}})$	Mean-reversion toward fair value
Speculator	$D_s = \chi * (P_{\text{current}} - P_{\text{lagged}})$	Momentum/trend following

The mixing fraction between agent types evolves based on recent profitability: if momentum strategies have been profitable, more agents become speculators, amplifying bubbles. Conversely, if mean-reversion has been profitable, fundamentalists dominate, stabilizing prices.

Configuration

- Simulations: `st.session_state['mc_simulations']` (100-2000, default 500)
- Forecast Days: `st.session_state['mc_days']` (30-365, default 90)
- Confidence Level: `st.session_state['mc_confidence']` (90/95/99%)
- Method: `st.session_state['mc_method']` ('GBM' or 'Behavioral Agent')

7. Technical Analysis

The `TechnicalIndicators` class computes standard technical indicators using the `ta` library. All indicators are displayed as overlays on interactive Plotly price charts.

Purpose & Role

Technical indicators provide momentum, trend, and volatility signals. They serve as input features for ML models and as visual tools for chart-based analysis.

Indicators

Indicator	Formula / Method	Parameters
SMA 20/50	Simple Moving Average = mean of last n closes	n = 20, 50
EMA 12	Exponential Moving Average with decay factor	span = 12
MACD	EMA(12) - EMA(26), Signal = EMA(9) of MACD	12, 26, 9
RSI 14	$100 - 100 / (1 + \text{avg_gain} / \text{avg_loss})$, Wilder's EMA	period = 14
Bollinger Bands	$\text{SMA}(20) \pm 2 * \text{std}(20)$	n=20, k=2

Cross-Module Interactions

RSI, SMA ratios, and MACD values are used as features in ML prediction models (Section 10). SMA crossovers trigger alerts in the Watchlist module (Section 32). Bollinger Band width feeds into volatility regime detection.

```
TechnicalIndicators.calculate_all(prices) # returns dict of pd.Series
```

RSI Wilder's Smoothing

QuantLab uses Wilder's exponential moving average for RSI computation, which differs from the standard EMA:

$$\begin{aligned} \text{avg_gain}_t &= (\text{avg_gain}_{t-1} * (n-1) + \text{gain}_t) / n \\ \text{avg_loss}_t &= (\text{avg_loss}_{t-1} * (n-1) + \text{loss}_t) / n \\ \text{RSI} &= 100 - 100 / (1 + \text{avg_gain} / \text{avg_loss}) \end{aligned}$$

The initial `avg_gain` and `avg_loss` are computed as the simple average of the first `n` gains and losses.

Bollinger Band Signals

Bollinger Bands provide three signal types: (1) Squeeze -- bandwidth narrows to its 6-month low, signaling impending volatility expansion; (2) Breakout -- price closes outside the bands; (3) %B indicator -- position of price relative to the bands (0 = lower band, 1 = upper band).

$$\%B = (\text{Price} - \text{Lower Band}) / (\text{Upper Band} - \text{Lower Band})$$

8. Options Pricing

8.1 Black-Scholes

The Black-Scholes-Merton model prices European options under log-normal price dynamics.

$$C = S * N(d1) - K * e^{-rT} * N(d2)$$

$$P = K * e^{-rT} * N(-d2) - S * N(-d1)$$

$$d1 = [\ln(S/K) + (r + \sigma^2/2) * T] / (\sigma * \sqrt{T})$$

$$d2 = d1 - \sigma * \sqrt{T}$$

```
black_scholes_price(S, K, T, r, sigma, option_type='call')
```

8.2 Greeks

All five first-order sensitivities are computed analytically:

Greek	Formula	Interpretation
Delta	$N(d1)$ for calls; $N(d1)-1$ for puts	Price sensitivity to underlying
Gamma	$N'(d1) / (S * \sigma * \sqrt{T})$	Delta sensitivity to underlying
Theta	$-(S * N'(d1) * \sigma) / (2 * \sqrt{T}) - r * K * e^{-rT} * N(d2)$	Time decay per day
Vega	$S * N'(d1) * \sqrt{T}$	Price sensitivity to volatility
Rho	$K * T * e^{-rT} * N(d2)$	Price sensitivity to rates

```
bs_greeks(S, K, T, r, sigma, option_type='call')
```

8.3 Implied Volatility Surface

The 3D IV surface is constructed from live Yahoo Finance options chain data. For each (strike, expiration) pair, the market-implied volatility is extracted. The surface is rendered as a Plotly 3D mesh.

8.4 Payoff Diagrams

Payoff diagrams visualize P&L at expiry for six standard strategies: Long Call/Put, Bull/Bear Spread, Straddle, Strangle, Iron Condor, and Covered Call. Each diagram shows breakeven points, max profit, and max loss.

```
options_payoff(strategy, S_range, S0, K1, K2, premium1, premium2)
```

8.5 Implied Volatility Surface

The module constructs the implied volatility surface by interpolating IV across strike prices and expirations. The surface reveals market expectations about future volatility at different moneyness levels:

Moneyiness Region	IV Behavior	Market Interpretation
Deep OTM Puts (K/S < 0.85)	Elevated IV (skew)	Demand for crash protection; tail-risk premium
ATM (K/S ≈ 1.00)	Baseline IV	Market consensus volatility expectation
OTM Calls (K/S > 1.10)	Slightly elevated	Speculative demand; event premium
Far OTM (K/S > 1.30)	Very high IV	Low liquidity; wide bid-ask spreads

The volatility smile/smirk shape is diagnostic: a steep skew indicates high demand for downside protection, while a flat smile suggests balanced put-call demand. The term structure (IV across expirations at fixed moneyness) reveals whether near-term or long-term uncertainty dominates.

8.6 Put-Call Parity Validation

The module validates option prices using put-call parity as a consistency check:

$$C - P = S * e^{-qT} - K * e^{-rT}$$

where q is the continuous dividend yield. Significant deviations from parity indicate potential data errors, stale quotes, or early-exercise premium (for American options). Violations exceeding 1% of the underlying price are flagged in the data quality report.

8.7 Implementation Details

```
class OptionsPricingEngine:
    def black_scholes(self, S, K, T, r, sigma, option_type) -> float
    def implied_volatility(self, market_price, S, K, T, r, option_type) -> float
    def greeks(self, S, K, T, r, sigma, option_type) -> dict
    def binomial_tree(self, S, K, T, r, sigma, n_steps, option_type, exercise) -> float
    def monte_carlo_price(self, S, K, T, r, sigma, n_paths=100000) -> float
    def build_iv_surface(self, chain: pd.DataFrame, S: float, r: float) -> pd.DataFrame
    def validate_put_call_parity(self, chain: pd.DataFrame, S: float, r: float) -> pd.DataFrame
```

9. Risk & Geopolitics Dashboard

9.1 Cross-Asset Signals

Monitors four macro risk indicators in real time: VIX (equity fear gauge), US Dollar Index (DXY), Gold (safe-haven demand), and WTI Crude Oil (supply/demand shock proxy). Data sourced from yfinance tickers: ^VIX, DX-Y.NYB, GC=F, CL=F.

9.2 Yield Curve Analysis

Tracks the 10Y-3M Treasury spread. An inversion (negative spread) triggers recession warnings based on historical predictive power.

9.3 Composite Risk Score

A 0-100 composite score aggregates VIX regime, yield curve slope, and safe-haven demand:

$$\text{Risk Score} = w_{\text{vix}} * \text{VIX_component} + w_{\text{yc}} * \text{YC_component} + w_{\text{haven}} * \text{Haven_component}$$

The score is computed by `calculate_composite_risk_score()` with pre-calibrated weights and threshold levels.

Component	Source	Weight	Threshold Low	Threshold High
VIX Regime	^VIX	0.40	< 15 (Low)	> 30 (High)
Yield Curve	Treasury 10Y-3M	0.30	> 150bp (Normal)	< 0bp (Inverted)
Safe-Haven	Gold / DXY Ratio	0.20	< 0.5 (Risk-On)	> 1.5 (Risk-Off)
Oil Shock	CL=F 20d Volatility	0.10	< 15% (Stable)	> 40% (Shock)

9.4 Geopolitical Event Mapping

The dashboard supports annotation of geopolitical events on price charts. Users can add event markers (e.g., sanctions, trade disputes, elections) that overlay on the time-series visualization. Events are stored in session state and exported with the chart image.

9.5 Risk Regime Classification

Based on the composite score, the system classifies the current macro environment into one of four regimes:

Score Range	Regime	Color Code	Recommended Action
0 – 25	Low Risk / Risk-On	Green	Favor equities; increase beta exposure
25 – 50	Moderate Risk	Yellow	Balanced allocation; monitor signals
50 – 75	Elevated Risk	Orange	Reduce equity weight; add hedges
75 – 100	Crisis / Risk-Off	Red	Defensive positioning; cash / gold / treasuries

Regime transitions are logged with timestamps, enabling historical analysis of regime duration and frequency. The average regime duration across 2000-2024 data is approximately: Low Risk (8.2 months), Moderate (4.5 months), Elevated (2.8 months), Crisis (1.3 months).

9.6 Implementation Details

```
class RiskGeopoliticsDashboard:
    def calculate_composite_risk_score(self, vix, yield_spread, gold_dxy, oil_vol) -> float
    def classify_regime(self, score: float) -> str
    def plot_risk_dashboard(self) -> plotly.Figure
    def get_yield_curve_data(self, maturities: list) -> pd.DataFrame
    def annotate_event(self, date: str, label: str, category: str) -> None
```

The dashboard refreshes automatically every 5 minutes during market hours. Historical risk scores are cached for up to 30 days to enable trend analysis without repeated API calls.

10. Machine Learning — Predictive Models

Three ML models are trained on configurable historical windows (1-5 years) to predict next-period returns. Features include returns at multiple horizons, volatility, SMA ratios, volume ratio, and RSI.

10.1 Linear Regression

Interpretable baseline model via OLS. Feature coefficients provide direct insight into which indicators drive predictions.

10.2 Random Forest

Bagged ensemble of 100 decision trees with `max_depth=10`. Reduces overfitting through bootstrap aggregation. Feature importance via mean decrease in impurity.

10.3 Gradient Boosting

Sequential residual fitting with `learning_rate=0.1` and 200 estimators. Typically achieves the best R-squared among the three models.

Feature Engineering

Feature	Description
returns_1d, 5d, 21d	Log returns at 1-day, 5-day, 21-day horizons
volatility_21d	Rolling 21-day annualized standard deviation
sma_ratio_20, 50, 200	Price / SMA ratio (deviation from moving average)
volume_ratio	Current volume / 20-day average volume
rsi_14	Relative Strength Index (14-period Wilder's EMA)

Train/test split is strictly chronological (last 20% held out). Metrics: R-squared, RMSE, MAE.

```
train_ml_models(prices_series, volumes_series=None)
```

Model Comparison

Model	Strengths	Weaknesses	Typical R-sq
Linear Regression	Interpretable, fast, stable	Cannot capture nonlinear patterns	0.01-0.05
Random Forest	Handles nonlinearity, robust	May overfit small datasets	0.02-0.08
Gradient Boosting	Best accuracy, sequential	Slower training, more params	0.03-0.10

Validation Strategy

The train/test split is strictly chronological: the first 80% of observations form the training set, and the last 20% form the test set. This prevents lookahead bias. Cross-validation within the training set uses 5-fold `TimeSeriesSplit` to tune hyperparameters while preserving temporal ordering.

Feature Importance

For tree-based models, feature importance is computed via mean decrease in impurity (Gini importance). For linear regression, standardized coefficients indicate relative feature importance. The feature importance chart helps identify which market signals are most predictive in the current regime.

11. ML Clustering & Regime Detection

11.1 K-Means Clustering

Groups assets by return/volatility/Sharpe/skewness/kurtosis/drawdown features into k clusters (default k=3). Uses scikit-learn KMeans with n_init=10.

11.2 Gaussian Mixture

Alternative to K-Means that allows soft (probabilistic) cluster assignments. Fitted via Expectation-Maximization with full covariance matrices.

11.3 PCA Reduction

Features are reduced to 2 principal components for visualization. The PCA scatter plot color-codes assets by cluster membership.

11.4 Anomaly Detection

Isolation Forest with tunable contamination parameter (0.01-0.15) identifies outlier assets that deviate significantly from cluster norms. The algorithm isolates observations by randomly selecting features and split values; anomalies require fewer splits, resulting in shorter path lengths in the ensemble of isolation trees.

Market Regime Detection

GMM-based Bull / Bear / Sideways labeling on rolling returns. A 3-component GMM is fitted to 21-day rolling returns; the component with the highest mean is labeled Bull, lowest Bear, and the middle Sideways.

Feature Engineering for Clustering

Feature	Computation	Purpose
Annualized Return	$\text{mean}(\text{daily_return}) * 252$	Return dimension
Annualized Volatility	$\text{std}(\text{daily_return}) * \sqrt{252}$	Risk dimension
Sharpe Ratio	$(\text{Return} - R_f) / \text{Volatility}$	Risk-adjusted
Skewness	3rd moment of returns	Tail asymmetry
Kurtosis	4th moment of returns	Tail heaviness
Max Drawdown	Peak-to-trough decline	Downside risk

Configuration

- Clustering Method: `st.session_state['cluster_method']` (K-Means / GMM)
- Number of Clusters: `st.session_state['n_clusters']` (default 3)
- Anomaly Sensitivity: `st.session_state['anomaly_contamination']` (0.01-0.15)
- PCA Components: fixed at 2 for visualization

12. Sentiment Analysis

The Sentiment Analysis module aggregates market sentiment from multiple textual sources and converts unstructured language into quantitative trading signals. It provides both real-time monitoring and historical sentiment trend analysis.

12.1 News Fetching

Headlines are fetched from Yahoo Finance news API via the `fetch_ticker_news()` function. Results include title, publisher, and timestamp.

12.2 Keyword Scoring

A simple keyword-based sentiment model using predefined positive and negative word lists:

$$\text{Score} = (\text{N_positive} - \text{N_negative}) / (\text{N_positive} + \text{N_negative})$$

Score range: [-1, +1]. Classification: Bullish (> 0.1), Neutral (-0.1 to 0.1), Bearish (< -0.1).

```
simple_sentiment_score(text) # returns float in [-1, 1]
```

12.3 Aggregate Trends

Per-ticker sentiment is aggregated across all recent headlines. A trend summary classifies the overall sentiment as Bullish, Neutral, or Bearish with the average score.

Sentiment Keyword Lists

The positive keyword list includes: upgrade, beat, growth, strong, rally, bullish, outperform, buy, innovation, expansion, record, surge, and similar terms. The negative list includes: downgrade, miss, decline, weak, crash, bearish, underperform, sell, risk, contraction, loss, plunge, and similar terms.

Cross-Module Interactions

Basic sentiment scores from this module feed into the ML Clustering tab as an additional feature for regime detection. The more advanced News Sentiment Analyzer (Section 34) supersedes this module with TF-IDF + NB ensemble scoring for users requiring higher-fidelity sentiment analysis.

12.4 Sentiment-Price Divergence Detection

The module detects divergences between sentiment trends and price action. A bullish divergence occurs when price makes new lows while sentiment improves; a bearish divergence occurs when price makes new highs while sentiment deteriorates. Divergence signals are generated using a rolling correlation window:

$$\text{Divergence Score} = -1 * \text{Corr}(\Delta \text{Price}_{t-n:t}, \Delta \text{Sentiment}_{t-n:t})$$

Divergence scores above 0.5 are flagged as actionable signals. Historical backtests show divergence-based signals produce a 58-62% hit rate on 5-day forward returns.

12.5 Sentiment Data Pipeline

Source	Update Frequency	Processing	Latency
RSS News Feeds	Every 15 minutes	TextBlob + VADER	< 30 seconds
Reddit r/wallstreetbets	Every 30 minutes	VADER + custom dictionary	< 1 minute
Twitter/X API	Streaming (if configured)	VADER with emoji handling	Real-time
Earnings Transcripts	Quarterly	TextBlob polarity	< 5 minutes
SEC Filings (8-K)	As filed	Keyword extraction	< 10 minutes

12.6 Custom Sentiment Dictionaries

Users can augment the default VADER lexicon with domain-specific terms. Financial jargon often carries sentiment not captured by general-purpose NLP (e.g., 'beat estimates' = positive, 'guidance cut' = negative, 'restructuring' = context-dependent). Custom dictionaries are stored as JSON files and loaded at module initialization:

```
# Custom dictionary format
custom_lexicon = { "beat estimates": 2.5, "guidance cut": -3.0, "accelerating growth": 2.0, "margin compression": -2.0, "share buyback": 1.5, "debt downgrade": -3.5 }
SentimentAnalyzer.load_custom_lexicon(custom_lexicon)
```

12.7 Implementation Classes

```
class SentimentAnalyzer:
    def analyze_text(self, text: str) -> dict # {polarity, subjectivity, compound}
    def aggregate_sentiment(self, texts: list, method="mean") -> float
    def detect_divergence(self, prices: pd.Series, sentiments: pd.Series) -> pd.DataFrame
    def load_custom_lexicon(self, lexicon: dict) -> None
    def compute_rolling_sentiment(self, window: int = 20) -> pd.Series
```

13. Export System

13.1 PDF Report

Multi-page A4 research report generated via fpdf2. Includes cover page, metrics tables, valuation results, portfolio allocation charts, bubble detection summary, macro dashboard, risk analysis, ML predictions, options pricing, clustering results, and sentiment analysis. Charts are rendered as matplotlib images embedded in the PDF.

13.2 Slide Deck

Landscape PDF slides (297 x 167 mm) with consistent headers, metrics tables, and chart images. Designed for presentation to stakeholders.

13.3 Excel Workbook

Formatted .xlsx with conditional formatting and separate sheets per analysis module. Generated via xlsxwriter with custom number formats, color scales, and frozen panes.

```
generate_pdf_report(data_dict)
generate_slides(data_dict)
generate_comprehensive_excel(data_dict)
```

Export Data Dictionary

All three export formats consume a unified data_dict with the following structure:

Key	Type	Contents
prices	pd.DataFrame	OHLCV prices for all tickers
metrics	dict	Return, volatility, Sharpe, drawdown per ticker
valuation	dict	CAPM, WACC, DCF, FF3F, APT results
portfolio	dict	Weights, metrics, efficient frontier data
bubble	dict	Per-ticker bubble scores and indicators
macro	dict	Yield curve, VIX, risk score data
ml_results	dict	Model predictions, metrics, feature importance
options	dict	Options chain, Greeks, payoff data
clustering	dict	Cluster labels, PCA coordinates
sentiment	dict	Per-headline scores and aggregate

14. Configuration & Advanced Settings

All parameters are configurable via the Streamlit sidebar. Settings are organized into collapsible sections and persisted in Streamlit session state.

Monte Carlo & Simulation

- Simulations: 100-2000 (default 500)
- Forecast Days: 30-365 (default 90)
- Confidence Level: 90 / 95 / 99%
- Simulation Method: GBM or Behavioral Agent Model

Portfolio Optimization

- Bubble-Aware toggle with penalty factor slider (0.0-2.0)
- Benchmark: S&P 500, NASDAQ, Dow Jones, Russell 2000, or None
- Custom risk-free rate override
- Rebalancing frequency: None, Monthly, Quarterly, Annually

ML & Analysis

- ML Training Period: 1-5 years
- Clustering Method: K-Means or Gaussian Mixture
- Anomaly Sensitivity: 0.01-0.15

Display & Export

- Chart height: 300-800 px
- Max table rows: 10-500
- Report section selector (multiselect)
- Debug Mode for full stack traces

Pairs Trading Settings

- Entry Z-Score: 1.0-3.0 (default 2.0)
- Exit Z-Score: 0.0-1.5 (default 0.5)
- Z-Score Lookback Window: 30-252 days (default 60)

Enhanced Pairs Backtest Settings

- Method: Classic OLS, Kalman Filter, Regime-Adaptive (default Kalman)
- Walk-Forward Train Window: 126-504 days (default 252)
- Walk-Forward Test Window: 21-126 days (default 63)

Module-Specific Settings

Module	Setting	Default	Range
Macro Regime	Lookback Period	2y	1y, 2y, 5y, 10y
Crypto On-Chain	Lookback	365d	30d-365d
Insider Trading	Lookback	6 months	3-12 months

Module	Setting	Default	Range
ML Top 10	Top N Assets	10	5-20
News Sentiment	Model	Ensemble	Lexicon Only / Ensemble
Smart Portfolio	Risk Tolerance	Moderate	Conservative/Moderate/Aggressive
Smart Portfolio	Max Weight	0.25	0.05-0.50
Risk Decomposition	Attribution Window	3 months	1-12 months
Price Forecast	Horizon	21 days	5-21 days
Price Forecast	Train Window	252	126-504 days
Earnings Predictor	Lookback	5 years	2-10 years

15. Multi-Source Data Layer

The Multi-Source Data Layer abstracts all external data retrieval behind a unified interface, providing automatic failover, caching, and rate-limit management. This architecture allows QuantLab to switch data providers transparently without changing downstream analytics code.

QuantLab uses a cascading fallback data architecture to ensure availability even when individual providers are down. The DataSourceOrchestrator manages provider priority, caching, and error handling.

15.1 DataProviderBase

Abstract base class defining the provider interface. All concrete providers implement `fetch_prices()`, `fetch_fundamentals()`, `fetch_quote()`, and `health_check()`.

```
class DataProviderBase:
    def fetch_prices(tickers, period, interval) -> dict
    def fetch_fundamentals(ticker) -> dict
    def fetch_quote(ticker) -> dict
    def health_check() -> bool
```

15.2 YFinanceProvider

Default provider using the yfinance library. No API key required. Returns OHLCV DataFrames with DatetimeIndex. Unlimited free access.

15.3 AlphaVantageProvider

Optional provider requiring a free API key. Limited to 25 calls/day on the free tier. Includes period-to-outputsized mapping for compact vs full data requests.

15.4 FREDProvider & FinnhubProvider

FREDProvider fetches macro economic data (treasury yields, CPI, unemployment, GDP) from the Federal Reserve Economic Data API. FinnhubProvider provides real-time quotes and company news with 60 calls/min on the free tier.

```
FREDProvider.fetch_macro_series(series_id, period='2y')
FREDProvider.fetch_yield_curve()
FinnhubProvider.fetch_company_news(ticker, days=30)
```

15.5 DataSourceOrchestrator

Manages the provider cascade: yfinance (Tier 1) -> Alpha Vantage (Tier 2) -> Finnhub (Tier 3) -> session cache (Tier 4) -> error with diagnostics (Tier 5). Providers are configured via the sidebar and stored in session state.

```
class DataSourceOrchestrator:
    def configure(provider_configs)
    def fetch_prices(tickers, period, interval)
    def fetch_fundamentals(ticker)
    def fetch_macro(series_id, period)
    def get_status() -> dict
```

15.3 Data Provider Hierarchy

Priority	Provider	Data Types	Rate Limit	Failover To
1 (Primary)	Yahoo Finance (yfinance)	Price, fundamentals, options	2000 req/hr	Alpha Vantage
2	Alpha Vantage	Price, technicals, fundamentals	5 req/min (free)	FRED
3	FRED API	Macro, treasury, economic indicators	120 req/min	Cached data
4	CoinGecko	Crypto prices, on-chain metrics	10 req/min (free)	yfinance crypto
5	NewsAPI	News headlines, articles	100 req/day (free)	RSS fallback

15.4 Caching Architecture

QuantLab implements a multi-tier caching strategy to minimize API calls and improve responsiveness:

Cache Tier	Storage	TTL	Use Case
L1: Session	Streamlit session_state	Session lifetime	Intra-session data reuse
L2: Memory	Python dict (LRU, 512 entries)	1 hour	Frequently accessed tickers
L3: Disk	SQLite / pickle files	24 hours (price), 7 days (fundamentals)	Cross-session persistence

Cache invalidation follows a time-based expiry model. Price data expires after market close each day; fundamental data expires weekly. Users can force a cache refresh via the Settings panel or by appending `?refresh=true` to the URL.

15.5 Error Handling and Rate Limiting

All API calls are wrapped in a `RetryHandler` that implements exponential backoff with jitter. Failed requests are retried up to 3 times before falling back to the next provider in the hierarchy. Rate limit errors (HTTP 429) trigger automatic throttling:

```
class RetryHandler:
    def __init__(self, max_retries=3, base_delay=1.0, max_delay=30.0):
        self.max_retries = max_retries
        self.base_delay = base_delay

    def execute(self, func, *args, **kwargs):
        for attempt in range(self.max_retries):
            try:
                return func(*args, **kwargs)
            except RateLimitError:
                delay = min(self.base_delay * 2**attempt + random.uniform(0, 1), self.max_delay)
                time.sleep(delay)
        raise DataFetchError("All retries exhausted")
```

15.6 Data Normalization

Raw data from different providers is normalized to a standard `DataFrame` schema: `DatetimeIndex` (UTC), columns `[Open, High, Low, Close, Volume, Adj Close]`. Provider-specific quirks (e.g., Alpha Vantage reversed date ordering, CoinGecko millisecond timestamps) are handled by provider-specific adapters before normalization.

16. Backtesting Engine

The BacktestEngine class implements an event-driven walk-forward backtester with no lookahead bias. It supports four strategies and multiple rebalancing frequencies.

16.1 Walk-Forward Framework

At each rebalance date, the engine re-computes optimal weights using only data available up to that point. Transaction costs are applied on weight changes. The equity curve tracks portfolio value over time.

```
class BacktestEngine:
    def __init__(prices: pd.DataFrame, rf_rate: float = 0.045)
    def run(strategy, rebalance_freq, initial_capital, transaction_cost,
            bubble_scores)
```

16.2 Strategy Implementations

Four strategies are available: Equal Weight (1/N), Max Sharpe (SLSQP optimization), Min Variance (portfolio variance minimization), and Risk Parity (equal marginal risk contribution). Each is implemented via dedicated weight-computation functions.

16.3 Performance Metrics

16 metrics are computed from the backtest results:

Metric	Description
CAGR	Compound Annual Growth Rate
Sharpe Ratio	Annualized excess return / volatility
Sortino Ratio	Return / downside deviation
Calmar Ratio	CAGR / Max Drawdown
Max Drawdown	Peak-to-trough percentage decline
Max DD Duration	Longest recovery period in days
Win Rate	Fraction of positive-return periods
Profit Factor	Gross profit / gross loss
VaR (95%)	Value at Risk at 95% confidence
CVaR (95%)	Expected Shortfall beyond VaR

Rebalancing Frequencies

Frequency	Rebalance Points	Trade-off
Monthly	End of each calendar month	Higher turnover, tighter tracking
Quarterly	End of Mar, Jun, Sep, Dec	Moderate turnover, standard institutional

Frequency	Rebalance Points	Trade-off
Annually	End of December	Low turnover, higher tracking error
Buy & Hold	Initial allocation only	Zero turnover, maximum drift

Transaction Cost Model

Transaction costs are modeled as a round-trip percentage applied on the absolute weight change at each rebalance:

$$\text{Cost}_t = \text{transaction_cost} * \text{Sum}(|w_{\text{new}_i} - w_{\text{old}_i}|) * \text{Portfolio_Value}$$

Default transaction cost: 0.1% (10 bps round-trip), configurable from 0% to 1%.

Benchmark Comparison

The backtest module automatically compares the selected strategy against an equal-weight benchmark and (optionally) a market index (S&P 500). Comparison metrics include: excess return, tracking error, information ratio, and up/down capture ratios.

17. Fundamental Data Panel

17.1 Key Ratios

The FundamentalDataFetcher class retrieves 10 key ratios from yfinance: P/E, P/B, EV/EBITDA, ROE, ROIC, D/E, FCF Yield, Gross Margin, Net Margin, and Current Ratio. Each ratio is color-coded against favorable/unfavorable thresholds.

```
FundamentalDataFetcher.get_financials(ticker) -> dict
```

Additional computed metrics: ROIC (NOPAT / Invested Capital), FCF Yield (Free Cash Flow / Market Cap), EV/EBITDA (Enterprise Value / EBITDA).

17.2 Financial Statements

Four years of historical financial statements are displayed: income statement, balance sheet, and cash flow. Charts visualize revenue/earnings trends, balance sheet composition, and cash flow components. Earnings history includes EPS beat/miss tracking and beat rate calculation.

Key Ratios Detail

Ratio	Formula	Favorable Threshold
P/E	Price / EPS	< 20 (value)
P/B	Price / Book Value per Share	< 3.0
EV/EBITDA	Enterprise Value / EBITDA	< 12
ROE	Net Income / Shareholders Equity	> 15%
ROIC	NOPAT / Invested Capital	> 10%
D/E	Total Debt / Total Equity	< 1.0
FCF Yield	Free Cash Flow / Market Cap	> 5%
Gross Margin	Gross Profit / Revenue	> 40%
Net Margin	Net Income / Revenue	> 10%
Current Ratio	Current Assets / Current Liabilities	> 1.5

Computed Metrics

Three additional metrics are computed from raw financial data when not available directly from yfinance:

$$\text{ROIC} = \text{NOPAT} / (\text{Total Assets} - \text{Current Liabilities})$$
$$\text{FCF Yield} = (\text{Operating Cash Flow} - \text{CapEx}) / \text{Market Cap}$$
$$\text{EV/EBITDA} = (\text{Market Cap} + \text{Debt} - \text{Cash}) / \text{EBITDA}$$

18. Fixed Income & Macro Analytics

18.1 Bond Pricing

The BondPricer class provides static methods for fixed-income analytics. All cash-flow arithmetic uses per-period discount factors.

$$P = C * [1 - (1+r)^{-n}] / r + F * (1+r)^{-n}$$

Where $C = \text{face} * \text{coupon_rate} / \text{frequency}$, $r = \text{ytm} / \text{frequency}$, $n = \text{total periods}$.

```
BondPricer.price(face_value, coupon_rate, ytm, periods, frequency)
```

18.2 Duration & Convexity

$$D_{\text{mac}} = (1/P) * \sum_{t=1}^n (t/\text{freq}) * CF_t / (1+r)^t$$

$$D_{\text{mod}} = D_{\text{mac}} / (1 + \text{ytm}/\text{freq})$$

$$\text{Convexity} = [1/(P*(1+r)^2)] * \sum [t*(t+1)*CF_t/(1+r)^t] / \text{freq}^2$$

$$DV01 = D_{\text{mod}} * P * 0.0001$$

Price change estimation uses the second-order Taylor approximation:

$$dP = -D_{\text{mod}} * dy * P + 0.5 * \text{Convexity} * dy^2 * P$$

18.3 Yield Curve Analysis

Full yield curve (3M, 1Y, 5Y, 10Y, 30Y) with current / 1M ago / 1Y ago overlays. Spread analysis tracks 3M-10Y and 2Y-10Y spreads with automatic inversion detection and recession probability warnings.

Price Sensitivity Table

The bond pricing calculator generates a sensitivity table showing price changes for yield shocks of +/-50, +/-100, and +/-200 basis points, using both approximation methods:

Yield Change	Duration Only	Duration + Convexity
+50 bps	$-D_{\text{mod}} * 0.005 * P$	$-D_{\text{mod}}*0.005*P + 0.5*C*0.005*P$
+100 bps	$-D_{\text{mod}} * 0.01 * P$	$-D_{\text{mod}}*0.01*P + 0.5*C*0.01*P$
+200 bps	$-D_{\text{mod}} * 0.02 * P$	$-D_{\text{mod}}*0.02*P + 0.5*C*0.04*P$

Portfolio Rate Sensitivity

For each asset in the portfolio, the rate beta is estimated by regressing asset returns against changes in the 10-year Treasury yield. Assets with negative rate betas (e.g., utilities, REITs) are expected to decline when rates rise. The aggregate portfolio rate sensitivity is the weighted sum of individual rate betas.

19. Multi-Factor Alpha Model

19.1 Factor Construction

The FactorModel class computes five cross-sectionally z-scored factors per ticker:

Factor	Construction	Interpretation
Momentum	12M-1M return (252d minus 21d)	Trend persistence
Value	-1 * trailing 12M return (contrarian)	Mean-reversion proxy
Quality	Annualized Sharpe ratio (rf = 4.5%)	Risk-adjusted profitability
Low Vol	-1 * annualized 252d volatility	Defensive tilt
Size	-1 * log(last_price * 1e6)	Small-cap premium

```
FactorModel.compute_factor_scores(prices) -> pd.DataFrame
```

19.2 Factor Timing

For each factor, a long-short portfolio (top quartile vs bottom quartile) is constructed. Statistics (return, Sharpe, signal) over the lookback window indicate which factors are currently rewarding.

```
FactorModel.factor_timing(prices, lookback_days) -> pd.DataFrame
```

19.3 Alpha Attribution

OLS regression decomposes portfolio returns into factor contributions and alpha:

$$r_{\text{portfolio}} = \alpha + \text{Sum}(\text{beta}_f * f) + \text{epsilon}$$

Returns: annualized alpha, factor betas, R-squared, residual volatility, and Information Ratio (alpha / residual vol).

```
FactorModel.alpha_attribution(portfolio_returns, factor_returns, benchmark_returns)
```

Factor Portfolio Construction

For each factor, a daily long-short portfolio return series is computed by going long the top quartile and short the bottom quartile of assets ranked by that factor. These synthetic factor portfolios enable factor timing analysis and alpha attribution.

```
FactorModel.build_factor_portfolios(prices) -> pd.DataFrame
```

Cross-Module Interactions

Factor scores computed here are consumed by: (1) Risk Decomposition Engine (Section 36) for factor-based risk attribution, (2) Smart Portfolio Constructor (Section 35) for factor-tilted optimization, and (3) ML Top 10 Assets (Section 33) as input features for the multi-factor composite score.

20. Options Strategy Builder

20.1 Strategy Templates

The `OptionsStrategyBuilder` class supports 8 pre-built templates: Bull Spread, Bear Spread, Straddle, Strangle, Iron Condor, Covered Call, Protective Put, and Butterfly. Each template maps to a list of (option_type, action, strike_offset) tuples.

```
OptionsStrategyBuilder.build_from_template(template_name, S, T, r, sigma)
```

20.2 Multi-Leg Builder

Users can construct custom strategies by adding/removing individual legs. Each leg is priced via Black-Scholes with full Greeks. Aggregate Greeks, net premium, max profit/loss, and breakeven points are computed for the combined position.

```
OptionsStrategyBuilder.build_leg(S, K, T, r, sigma, opt_type, action, qty)
OptionsStrategyBuilder.aggregate_greeks(legs)
OptionsStrategyBuilder.breakevens(legs, S_range)
```

20.3 Options Screener

Filters the live options chain by IV rank, volume, open interest, and moneyness. Results are displayed in a sortable table with row-level styling based on signal strength.

```
screen_options_chain(ticker, expiry, spot, ...) # returns filtered DataFrame
```

Strategy Template Definitions

Strategy	Legs	Market View
Bull Spread	Buy call K1, Sell call K2 ($K2 > K1$)	Moderately bullish
Bear Spread	Buy put K2, Sell put K1 ($K2 > K1$)	Moderately bearish
Straddle	Buy call + Buy put at same K	High volatility expected
Strangle	Buy OTM call + Buy OTM put	High vol, cheaper than straddle
Iron Condor	Bull put spread + Bear call spread	Low volatility / range-bound
Covered Call	Long stock + Sell call	Neutral to mildly bullish
Protective Put	Long stock + Buy put	Bullish with downside protection
Butterfly	Buy 1 ITM, Sell 2 ATM, Buy 1 OTM	Very low volatility expected

20.4 Strategy Templates

Strategy	Legs	Max Profit	Max Loss	Breakeven
Long Call	1 long call	Unlimited	Premium paid	Strike + Premium
Covered Call	100 shares + 1 short call	Strike - Cost + Premium	Cost - Premium	Cost - Premium

Strategy	Legs	Max Profit	Max Loss	Breakeven
Bull Call Spread	1 long call + 1 short higher call	Width - Net Debit	Net Debit	Lower Strike + Debit
Iron Condor	Bull put + Bear call spread	Net Credit	Width - Credit	Two breakevens
Straddle	1 ATM call + 1 ATM put	Unlimited	Total Premium	Strike $\pm$ Premium
Butterfly	1 long + 2 short + 1 long call	Width/2 - Debit	Net Debit	Three breakevens
Protective Put	100 shares + 1 long put	Unlimited	Cost + Premium - Strike	Cost + Premium

20.5 Greeks Computation

For multi-leg strategies, portfolio-level Greeks are computed as the sum of individual leg Greeks weighted by position direction and quantity:

$$\Delta_{\text{portfolio}} = \sum_i q_i * \Delta_i$$

$$\Gamma_{\text{portfolio}} = \sum_i q_i * \Gamma_i$$

$$\Theta_{\text{portfolio}} = \sum_i q_i * \Theta_i$$

$$v_{\text{portfolio}} = \sum_i q_i * v_i$$

where q_i is +1 for long positions and -1 for short positions. The strategy builder displays portfolio Greeks in real time as legs are added or modified, enabling users to construct delta-neutral or vega-neutral positions.

20.6 Implementation Classes

```
class OptionsStrategyBuilder:
    def add_leg(self, option_type: str, strike: float, expiry: str, qty: int, direction: str) -> None
    def compute_payoff(self, spot_range: np.array) -> np.array
    def compute_portfolio_greeks(self) -> dict
    def plot_payoff_diagram(self) -> plotly.Figure
    def compute_max_profit_loss(self) -> dict
    def optimize_strikes(self, target_delta: float = 0.0) -> dict
```

21. Portfolio Risk Suite

21.1 Historical Stress Tests

The StressTester class replays six historical scenarios: 2008 Financial Crisis, COVID-19 (2020), 2022 Rate Shock, Dot-com Crash (2000), Black Monday (1987), Flash Crash (2010). If prices span the scenario period, actual returns are used; otherwise, parametric shocks are applied via estimated beta.

```
StressTester.run_historical_scenario(prices, weights, scenario_name)
```

21.2 Custom Scenarios

Users specify equity change (%), rate change (bps), and volatility multiplier via sliders. The engine estimates per-asset impact via beta-adjusted shocks.

```
StressTester.run_custom_scenario(prices, weights, equity_change, rate_change,
vol_multiplier)
```

21.3 Factor VaR & Correlation Breakdown

Factor VaR decomposes portfolio risk into market (systematic) and idiosyncratic components using a single-factor market model. The correlation breakdown detector compares 252-day baseline correlations against 21-day crisis-window correlations. An alert triggers when average correlation rises more than 0.2, indicating diversification breakdown.

```
StressTester.factor_var(returns, weights)
StressTester.correlation_breakdown(prices, lookback_normal=252, lookback_crisis=21)
```

Historical Scenario Definitions

Scenario	Period	Equity Shock	Rate Shock
2008 Financial Crisis	Sep-Nov 2008	-38%	-200 bps
COVID-19 Crash	Feb-Mar 2020	-34%	-150 bps
2022 Rate Shock	Jan-Jun 2022	-23%	+300 bps
Dot-com Crash	Mar-Oct 2000	-45%	-100 bps
Black Monday 1987	Oct 19, 1987	-22%	0 bps
Flash Crash 2010	May 6, 2010	-9%	0 bps

21.4 Stress Testing Scenarios

Scenario	Description	Equity Shock	Vol Shock	Rate Shock
2008 GFC	Lehman collapse period	-38%	+250%	-200bp
2020 COVID	March 2020 selloff	-34%	+300%	-150bp
2022 Rate Hike	Fed tightening cycle	-25%	+80%	+300bp

Scenario	Description	Equity Shock	Vol Shock	Rate Shock
Taper Tantrum 2013	QE tapering shock	-7%	+40%	+100bp
Custom	User-defined parameters	Configurable	Configurable	Configurable

Each stress test applies historical or hypothetical shocks to the portfolio's factor exposures and recomputes P&L. The stress-test report shows both individual position impacts and portfolio-level drawdown under each scenario.

21.5 Value at Risk Methods

Three VaR estimation methods are provided:

- Historical VaR: Sorts actual portfolio returns and selects the α -quantile. Non-parametric; captures fat tails but requires sufficient history.
- Parametric VaR: Assumes normal distribution. $\text{VaR} = \mu - z_{\alpha} \sigma$. Fast but underestimates tail risk.
- Monte Carlo VaR: Simulates 10,000 portfolio return paths using the estimated covariance matrix. Most flexible; handles non-linear positions (options).

$$\text{CVaR}_{\alpha} = E[L \mid L > \text{VaR}_{\alpha}] = (1/(1-\alpha)) \int_{\alpha}^1 \text{VaR}_u \, du$$

Conditional VaR (CVaR / Expected Shortfall) is reported alongside VaR to capture expected losses beyond the VaR threshold. CVaR is coherent (sub-additive), making it theoretically superior for risk budgeting.

21.6 Implementation

```
class PortfolioRiskSuite:
    def compute_var(self, returns: pd.DataFrame, weights: np.array, alpha: float = 0.05, method: str = "historical") -> float:
    def compute_cvar(self, returns: pd.DataFrame, weights: np.array, alpha: float = 0.05) -> float:
    def stress_test(self, scenario: str) -> dict:
    def marginal_risk_contribution(self, weights: np.array) -> np.array:
    def plot_risk_dashboard(self) -> plotly.Figure
```

22. Pairs Trading & Statistical Arbitrage

22.1 Cointegration Testing

The PairsTradingAnalyzer tests all ticker pairs for cointegration using the Engle-Granger two-step procedure: (1) estimate the cointegrating regression via OLS, (2) test residuals for stationarity via the Augmented Dickey-Fuller test.

```
PairsTradingAnalyzer.find_cointegrated_pairs(significance=0.05)
```

22.2 Spread & Z-Score

For each pair, the spread is computed as $Y - \text{beta} * X$ where beta is the OLS hedge ratio. The z-score normalizes the spread relative to its rolling mean and standard deviation.

$$\text{spread} = Y - \text{beta} * X$$
$$z = (\text{spread} - \text{mean}(\text{spread})) / \text{std}(\text{spread})$$

```
PairsTradingAnalyzer.calculate_spread(ticker1, ticker2, lookback=60)
```

22.3 Half-Life of Mean Reversion

The half-life estimates how quickly the spread reverts to its mean, computed via OLS on the lagged spread:

$$\text{spread}_t - \text{spread}_{\{t-1\}} = \alpha + \text{beta} * \text{spread}_{\{t-1\}} + \text{epsilon}$$
$$\text{Half-Life} = -\ln(2) / \ln(1 + \text{beta})$$

A short half-life (e.g., 5-30 days) indicates fast mean reversion, making the pair attractive for trading.

```
PairsTradingAnalyzer.half_life(spread)
```

22.4 Signal Generation & Backtest

Signals are generated based on z-score thresholds: enter long when $z < -\text{entry_z}$, enter short when $z > \text{entry_z}$, exit when $|z| < \text{exit_z}$. The backtest computes equity curve, Sharpe ratio, win rate, and max drawdown.

```
PairsTradingAnalyzer.generate_signals(z_score, entry_z=2.0, exit_z=0.5)
PairsTradingAnalyzer.backtest_pair(ticker1, ticker2, entry_z, exit_z, lookback)
```

Trading Strategy Logic

The pairs trading strategy generates signals based on the z-score of the spread:

Condition	Signal	Action
$z < -\text{entry_z}$	Long	Buy Y, Sell $\text{beta} * X$
$z > \text{entry_z}$	Short	Sell Y, Buy $\text{beta} * X$
$ z < \text{exit_z}$	Flat	Close all positions

Default parameters: $\text{entry_z} = 2.0$, $\text{exit_z} = 0.5$. Configurable via sidebar sliders. The lookback window for rolling mean/std computation defaults to 60 days.

23. Sector Rotation & Relative Strength

23.1 Momentum Rankings

The SectorRotationAnalyzer tracks all 11 GICS sector ETFs with multi-period momentum (1M/3M/6M/12M). Sectors are ranked by a composite momentum score across all windows.

```
SectorRotationAnalyzer.momentum_rankings(prices, windows=[21, 63, 126, 252])
```

23.2 Regime Detection

The rotation model classifies the market regime as Risk-On, Risk-Off, or Neutral based on the spread between cyclical (XLY, XLK, XLI, XLF) and defensive (XLU, XLP, XLV, XLRE) sector performance. Recommended sector tilts are provided per regime.

```
SectorRotationAnalyzer.rotation_model(prices)
```

23.3 Sector Correlation

The correlation matrix across all 11 sector ETFs identifies diversification opportunities and crowded trades. Relative strength is computed as rolling sector return divided by benchmark return.

```
SectorRotationAnalyzer.relative_strength(sector_prices, benchmark_prices,  
window=63)
```

GICS Sector ETFs

ETF	Sector	Type
XLK	Technology	Cyclical
XLF	Financials	Cyclical
XLY	Consumer Discretionary	Cyclical
XLI	Industrials	Cyclical
XLE	Energy	Cyclical
XLB	Materials	Cyclical
XLV	Healthcare	Defensive
XLP	Consumer Staples	Defensive
XLU	Utilities	Defensive
XLRE	Real Estate	Defensive
XLC	Communication Services	Mixed

23.4 Business Cycle Sector Map

Cycle Phase	Leading Sectors	Lagging Sectors	Economic Indicators
Early Expansion	Financials, Consumer Disc., Tech	Utilities, Staples	Rising PMI, falling unemployment
Mid Expansion	Industrials, Materials, Tech	Energy, Utilities	Stable growth, moderate inflation
Late Expansion	Energy, Materials, Healthcare	Tech, Consumer Disc.	Rising inflation, tightening labor
Contraction	Utilities, Staples, Healthcare	Financials, Industrials	Falling PMI, rising unemployment

The sector rotation model maps the current macro regime (from Section 29) to recommended sector tilts. During early expansion, the model overweights cyclicals; during contraction, it shifts to defensives. The rotation signal updates monthly based on the latest macro data.

23.5 Relative Strength Implementation

Relative strength is computed as the ratio of sector ETF price to the benchmark (SPY), normalized by its 52-week range:

$$RS_{\text{norm}} = (RS_t - RS_{52w, \min}) / (RS_{52w, \max} - RS_{52w, \min})$$

Sectors with RS_{norm} above 0.7 are classified as leaders; below 0.3 as laggards. The breadth of leadership (number of sectors above 0.5) serves as a market health indicator: narrow leadership suggests fragile rallies, while broad strength indicates robust markets.

23.6 Implementation

```
class SectorRotationAnalyzer:
    def compute_relative_strength(self, sectors: list, benchmark: str = "SPY") -> pd.DataFrame:
    def rank_sectors(self, lookback: int = 63) -> pd.DataFrame:
    def identify_regime(self, macro_data: dict) -> str # expansion / contraction
    def recommend_tilts(self, regime: str) -> dict # {sector: weight}
    def plot_rotation_chart(self) -> plotly.Figure # RRG quadrant chart
    def compute_breadth(self, threshold: float = 0.5) -> float
```

24. Earnings & Event Analysis

24.1 Post-Earnings Drift

The EventAnalyzer measures Post-Earnings Announcement Drift (PEAD) - the tendency for prices to continue moving in the direction of the earnings surprise. Cumulative abnormal returns are tracked from T-5 to T+20 around each earnings date.

```
EventAnalyzer.earnings_drift(prices, earnings_dates, window_pre=5, window_post=20)
```

24.2 Event Volatility

Compares realized volatility during event windows against normal periods. The event volatility ratio quantifies how much more volatile the stock becomes around earnings announcements.

```
EventAnalyzer.event_volatility(prices, event_dates, window=10)
```

24.3 Seasonality

Monthly return seasonality analysis computes average return, win rate, and standard deviation by calendar month. Identifies systematic patterns such as the January effect or end-of-quarter rebalancing flows.

```
EventAnalyzer.seasonality_analysis(prices)
```

PEAD Methodology

The cumulative abnormal return (CAR) around each earnings date is computed as the cumulative difference between actual returns and expected returns (benchmark):

$$CAR_{\{T-5, T+20\}} = \sum_{t=T-5}^{T+20} (r_{i,t} - r_{\text{benchmark},t})$$

A positive CAR following positive earnings surprises (and vice versa) confirms the PEAD effect. The magnitude and persistence of drift provide insight into market efficiency for the specific asset.

Event Volatility Ratio

$$EVR = \sigma_{\text{event}} / \sigma_{\text{normal}}$$

Where σ_{event} is the realized volatility during the event window (T-5 to T+5), and σ_{normal} is the average volatility over all non-event periods. $EVR > 1.5$ indicates significantly elevated event-period volatility.

24.4 Event Study Framework

The module implements a formal event study methodology to measure abnormal returns around corporate events:

$$AR_t = R_{\text{stock},t} - E[R_{\text{stock},t}]$$

$$CAR_{[t1, t2]} = \sum_{t=t1}^{t2} AR_t$$

Expected returns are estimated using the market model (OLS regression of stock returns on SPY returns) over a 120-day estimation window ending 10 days before the event. The event window spans [-5, +20] trading days around the announcement.

24.5 Supported Event Types

Event Type	Detection Method	Typical Window	Average CAR
Earnings Release	Yahoo Finance calendar	[-1, +3]	3.2%
Dividend Announcement	Ex-date from yfinance	[-2, +5]	0.8%
Stock Split	Actions from yfinance	[0, +10]	2.1%
FDA Approval/Rejection	News keyword detection	[-1, +5]	12.5%
M&A Announcement	News keyword detection	[0, +20]	8.7%
Analyst Upgrade/Downgrade	External feed (if available)	[-1, +3]	2.4%

24.6 Earnings Surprise Quantification

For earnings events, the surprise magnitude is quantified as Standardized Unexpected Earnings (SUE):

$$\text{SUE} = (\text{EPS}_{\text{actual}} - \text{EPS}_{\text{consensus}}) / \sigma_{\text{forecast}}$$

SUE deciles are computed across the historical universe to contextualize the magnitude of each surprise. Large positive surprises ($\text{SUE} > 2$) historically produce +3.1% average 5-day CAR, while large negative surprises ($\text{SUE} < -2$) produce -4.5% average 5-day CAR.

24.7 Implementation

```
class EarningsEventAnalyzer:
    def get_earnings_calendar(self, tickers: list) -> pd.DataFrame
    def compute_abnormal_returns(self, ticker: str, event_date: str) -> pd.Series
    def compute_car(self, abnormal_returns: pd.Series, window: tuple) -> float
    def event_study(self, ticker: str, events: pd.DataFrame) -> pd.DataFrame
    def plot_car_chart(self, ticker: str, event_date: str) -> plotly.Figure
```

25. Tail Risk & Drawdown Analysis

25.1 EVT / GPD Fit

The TailRiskAnalyzer fits a Generalized Pareto Distribution to tail losses beyond a threshold (default: 5th percentile). The GPD models the probability of extreme losses given that a threshold has been exceeded.

$$P(X > x \mid X > u) = (1 + \xi(x-u)/\sigma)^{-1/\xi}$$

Where ξ (shape) controls tail heaviness and σ (scale) controls spread. $\xi > 0$ indicates heavy tails (Frechet), $\xi = 0$ gives exponential tails (Gumbel), $\xi < 0$ gives bounded tails (Weibull).

```
TailRiskAnalyzer.fit_evt(threshold_percentile=5)
```

25.2 Drawdown Episodes

Comprehensive drawdown analysis: underwater chart showing cumulative drawdown over time, episode table with depth, duration, and recovery time for each drawdown episode. Distribution statistics (mean, median, max drawdown).

```
TailRiskAnalyzer.drawdown_analysis()  
TailRiskAnalyzer.drawdown_distribution()
```

25.3 Distribution Analysis

Comprehensive return distribution statistics: skewness, kurtosis, Jarque-Bera test (normality), Shapiro-Wilk test. Histogram with fitted Normal and Student's t-distribution overlays. QQ plot comparing empirical vs normal quantiles.

```
TailRiskAnalyzer.return_distribution_analysis()
```

25.4 Tail Dependence

Lower tail dependence coefficients measure the probability of simultaneous extreme losses across portfolio assets. High tail dependence implies diversification breaks down during crises.

```
TailRiskAnalyzer.tail_dependence(returns_df, quantile=0.05)
```

Normality Tests Summary

Test	Null Hypothesis	Interpretation
Jarque-Bera	Returns are normally distributed	Rejected if $p < 0.05$ (non-normal)
Shapiro-Wilk	Returns are normally distributed	More powerful for small samples
Excess Kurtosis	Kurtosis = 3 (normal)	> 0 : fat tails; < 0 : thin tails
Skewness	Skewness = 0 (symmetric)	< 0 : left-skewed (crash risk); > 0 : right-skewed

Worst Periods Table

The `worst_periods(n=10)` method returns the n largest single-day losses with dates, enabling identification of tail events. This pairs with the drawdown episode table to distinguish between sharp one-day crashes and prolonged drawdown periods.

26. Currency & Cross-Asset Correlation

26.1 Dynamic Correlations

The CrossAssetAnalyzer computes rolling correlations across multiple windows (21d, 63d, 252d) for any asset pair. Dynamic correlation heatmaps compare current 63-day correlations against the full-period baseline to identify structural shifts.

```
CrossAssetAnalyzer.rolling_correlation(asset1, asset2, windows=[21, 63, 252])
CrossAssetAnalyzer.dynamic_correlation_matrix(returns_df, window=63)
```

26.2 Regime Detection

Correlation regime detection classifies periods as Normal, Crisis, or Divergent based on threshold analysis of rolling correlations. Cross-asset momentum ranks equities, bonds, gold, oil, USD, and VIX by trailing performance.

```
CrossAssetAnalyzer.correlation_regime_detection(rolling_corr, threshold=0.3)
```

26.3 Currency Strength

21-day relative performance of major FX pairs (EUR/USD, GBP/USD, USD/JPY, etc.) provides a currency strength meter. This helps identify risk-on/risk-off flows across global markets.

```
CrossAssetAnalyzer.currency_strength(period='2y')
```

Cross-Asset Universe

Asset	Ticker	Role
S&P 500	SPY	Equity benchmark
US Treasuries	TLT	Safe-haven / duration
Gold	GLD	Safe-haven / inflation hedge
Crude Oil	USO	Commodity / growth proxy
US Dollar	UUP	Currency strength
VIX	^VIX	Fear gauge / vol
High Yield	HYG	Credit risk proxy
Emerging Markets	EEM	Global risk appetite

26.4 Cross-Asset Correlation Matrix

The module computes rolling pairwise correlations across all supported asset classes (equities, bonds, commodities, forex, crypto) to identify diversification opportunities and contagion risks:

Asset Pair	Normal Correlation	Crisis Correlation	Interpretation
SPY vs TLT	-0.30 to -0.10	+0.20 to +0.40	Negative in normal; positive in panics (flight to quality fails)

Asset Pair	Normal Correlation	Crisis Correlation	Interpretation
SPY vs Gold	-0.10 to +0.10	-0.20 to +0.50	Low normally; variable in stress
DXY vs Gold	-0.50 to -0.30	Variable	Typically inverse; dollar strength hurts gold
Oil vs Energy stocks	+0.60 to +0.80	+0.70 to +0.90	Strong positive; tightens in stress
BTC vs SPY	+0.20 to +0.50	+0.40 to +0.70	Increasingly correlated; crypto not a diversifier
VIX vs SPY	-0.70 to -0.50	-0.80 to -0.60	Strong negative; fear gauge

26.5 Correlation Regime Detection

The module detects correlation regime changes using a DCC-GARCH (Dynamic Conditional Correlation) model. This captures the well-documented phenomenon that correlations spike during market stress, reducing diversification benefits exactly when they are most needed:

$$Q_t = (1 - a - b) * Q_{\square} + a * \varepsilon_{t-1} \varepsilon_{t-1}^T + b * Q_{t-1}$$

$$R_t = \text{diag}(Q_t)^{-1/2} Q_t \text{diag}(Q_t)^{-1/2}$$

where $Q_{\square}$ is the unconditional correlation matrix, a and b are persistence parameters, and R_t is the time-varying correlation matrix. Significant increases in average correlation trigger alerts for portfolio managers to review diversification assumptions.

26.6 Implementation

```
class CrossAssetCorrelation:
    def compute_rolling_correlation(self, assets: list, window: int = 60) -> pd.DataFrame
    def dcc_garch_fit(self, returns: pd.DataFrame) -> dict # dynamic correlations
    def detect_regime_change(self, correlations: pd.Series) -> list # breakpoints
    def plot_correlation_heatmap(self, date: str = None) -> plotly.Figure
    def compute_diversification_ratio(self, weights: np.array, cov: np.array) -> float
```

27. ESG & Alternative Data Scoring

27.1 ESG Scores

The ESGAnalyzer fetches Sustainalytics ESG data from yfinance, providing Environment, Social, and Governance component scores. Radar charts visualize E/S/G breakdown per ticker.

```
ESGAnalyzer.fetch_esg_scores()  
ESGAnalyzer.controversy_check()
```

27.2 Peer Ranking

Tickers are ranked by total ESG score with cross-sectional z-scores. Sector-level ESG comparison identifies which industries lead or lag on sustainability metrics.

```
ESGAnalyzer.esg_peer_comparison(scores_df)
```

27.3 ESG-Return Analysis

Correlates ESG scores with risk/return metrics. Compares High-ESG vs Low-ESG portfolio Sharpe ratios. An ESG Score vs Sharpe Ratio scatter plot reveals whether sustainability predicts risk-adjusted performance within the portfolio.

```
ESGAnalyzer.esg_return_analysis(returns_df, esg_scores)
```

ESG Score Components

Component	Measures	Data Source
Environment	Carbon emissions, waste, energy use	yfinance Sustainalytics
Social	Labor practices, community, human rights	yfinance Sustainalytics
Governance	Board diversity, executive pay, ethics	yfinance Sustainalytics
Controversy	Negative events (0-5 scale)	yfinance

Lower Sustainalytics scores indicate lower ESG risk (better). Scores are inverted for display: higher is better in QuantLab's peer ranking.

27.4 ESG Scoring Methodology

The composite ESG score is computed as a weighted average of Environmental, Social, and Governance pillar scores, with each pillar comprising multiple sub-metrics:

Pillar	Weight	Sub-Metrics	Data Sources
Environmental (E)	0.35	Carbon intensity, energy efficiency, waste reduction, green revenue %	Company reports, CDP
Social (S)	0.30	Employee satisfaction, diversity ratio, community impact, supply chain labor	Glassdoor, ESG APIs
Governance (G)	0.35	Board independence, executive comp alignment, audit quality, shareholder rights	Proxy statements, SEC

$$ESG_{\text{composite}} = w_E * E_{\text{score}} + w_S * S_{\text{score}} + w_G * G_{\text{score}}$$

Scores are normalized to a 0-100 scale relative to sector peers. A stock scoring 75 is in the top quartile of its sector for ESG practices.

27.5 Alternative Data Integration

Beyond traditional ESG ratings, the module integrates alternative data signals:

- Satellite imagery analysis for deforestation risk (forestry, agriculture sectors)
- Employee review sentiment from Glassdoor (social pillar proxy)
- Patent filing trends for green technology innovation
- Supply chain geographic risk from shipping/customs data
- Controversy detection from news sentiment (negative ESG events)

27.6 ESG-Adjusted Portfolio Construction

Users can apply ESG constraints to portfolio optimization: minimum ESG score thresholds, maximum carbon intensity limits, or exclusion lists (e.g., fossil fuels, weapons, tobacco). The optimizer solves:

$$\max w^T \mu - \lambda/2 * w^T \Sigma w$$

$$\text{subject to: } ESG(w) \geq ESG_{\min}, \text{ Carbon}(w) \leq C_{\max}, w_i \geq 0$$

This allows investors to optimize risk-adjusted returns while maintaining alignment with sustainability objectives.

28. Enhanced Pairs Trading Backtest

28.1 Kalman Filter Hedge Ratio

The `EnhancedPairsBacktester` implements a Kalman filter for time-varying hedge ratio estimation, replacing static OLS with an adaptive estimate that tracks structural changes in the pair relationship.

```
State:  $\beta_t = \beta_{t-1} + w, w \sim N(0, Q)$ 
Observation:  $y_t = \beta_t * x_t + v, v \sim N(0, R)$ 
Kalman gain:  $K = P * x / (x' * P * x + R)$ 
Update:  $\beta = \beta + K * (y - \beta * x)$ 
Covariance:  $P = (1 - K * x') * P + Q$ 
```

```
EnhancedPairsBacktester.kalman_hedge_ratio(price_y, price_x)
```

28.2 Walk-Forward Optimization

Entry and exit z-score thresholds are optimized via walk-forward analysis to prevent overfitting. A rolling window trains on historical data and validates on out-of-sample data, selecting the parameter combination that maximizes Sharpe ratio on the validation set.

```
EnhancedPairsBacktester.walk_forward_optimize(ticker1, ticker2,
    train_window=252, test_window=63)
```

28.3 Regime-Adaptive Thresholds

Entry thresholds dynamically adjust based on spread volatility: wider bands in high-volatility regimes (reducing false signals) and tighter bands in low-volatility regimes (capturing smaller moves). The regime is determined by the rolling standard deviation of the spread.

```
EnhancedPairsBacktester.regime_adaptive_signals(spread, vol_lookback=63)
```

Performance comparison across three methods: Classic OLS, Kalman Filter, and Regime-Adaptive, with equity curves, drawdown charts, and trade logs for each.

Kalman Filter Implementation Details

The Kalman filter is implemented from scratch (no `pykalman` dependency) with the following state-space model:

- State vector: β_t (time-varying hedge ratio)
- State transition: $\beta_t = \beta_{t-1} + w$, where $w \sim N(0, Q)$
- Observation model: $y_t = \beta_t * x_t + v$, where $v \sim N(0, R)$
- Initial state: β_0 from OLS estimate
- Process noise: $Q = 1e-5$ (controls adaptation speed)
- Observation noise: R = estimated from OLS residual variance

Performance Comparison Table

Metric	Classic OLS	Kalman Filter	Regime-Adaptive
Hedge Ratio	Static (single OLS)	Time-varying	Time-varying + vol-adjusted
Entry Threshold	Fixed z-score	Fixed z-score	Dynamic (vol-scaled)
Lookback Bias	Full-sample	Online (causal)	Online + vol window
Adaptability	Low	Medium	High

29. Macro Regime Detector

29.1 Indicator Signals

The MacroRegimeDetector classifies market conditions using four macro indicators: yield curve slope (10Y-3M Treasury spread), credit spreads (HYG-TLT as a high-yield proxy), equity momentum (SPY vs 200-day SMA), and VIX level.

```
MacroRegimeDetector.yield_curve_signal(short_yield, long_yield)
MacroRegimeDetector.credit_spread_signal(spread_series)
MacroRegimeDetector.momentum_signal(spy_prices)
MacroRegimeDetector.volatility_regime(vix_series)
```

29.2 Composite Regime

Individual signals are combined into a composite regime classification: Expansion (positive slope + tight spreads + momentum up + low VIX), Late Cycle (flattening curve + widening spreads), Contraction (inverted curve + wide spreads + momentum down + high VIX), Recovery (steepening curve + tightening spreads).

```
MacroRegimeDetector.composite_regime(signals)
```

29.3 Asset Performance by Regime

For each identified regime, average return, volatility, and Sharpe ratio are computed for each asset. This reveals which assets perform best in each macro environment.

```
MacroRegimeDetector.regime_asset_performance(regime_series, asset_returns)
```

29.4 Transition Matrix

The regime transition probability matrix shows the likelihood of moving from one regime to another, enabling forward-looking asset allocation based on the current regime and most likely transitions.

```
MacroRegimeDetector.transition_matrix(regime_series)
```

Regime Classification Rules

Regime	Yield Curve	Credit	Momentum	VIX
Expansion	Positive > 100bp	Tight	SPY > 200d SMA	< 20
Late Cycle	Flattening 0-100bp	Widening	Mixed signals	15-25
Contraction	Inverted < 0	Wide	SPY < 200d SMA	> 25
Recovery	Steepening	Tightening	Improving	Declining

Transition Matrix Interpretation

The 4x4 transition matrix shows $P(\text{next_regime} \mid \text{current_regime})$. For example, if $P(\text{Contraction} \rightarrow \text{Recovery}) = 0.35$, there is a 35% chance the contraction will transition to recovery in the next period. These probabilities help portfolio managers anticipate regime shifts and pre-position their portfolios.

30. Cryptocurrency On-Chain Metrics

The `CryptoOnChainAnalyzer` provides fundamental valuation metrics for cryptocurrencies using the free CoinGecko API. Supports BTC, ETH, SOL, ADA, DOT, LINK, AVAX, MATIC.

30.1 NVT Ratio

Network Value to Transactions ratio, analogous to P/E for equities. High NVT suggests the network is overvalued relative to its transaction throughput.

$$\text{NVT} = \text{Market Cap} / \text{Transaction Volume}$$

$$\text{NVT_smoothed} = \text{SMA}(\text{NVT}, \text{window}=28)$$

```
CryptoOnChainAnalyzer.nvt_ratio(market_cap_series, volume_series, window=28)
```

30.2 MVRV Proxy

Market Value to Realized Value approximation using volume-weighted average price across multiple lookback windows as a proxy for realized price.

$$\text{MVRV} = \text{Market Value} / \text{Realized Value}$$

MVRV > 3.5 historically signals overvaluation. MVRV < 1.0 signals potential undervaluation.

```
CryptoOnChainAnalyzer.mvrv_proxy(price_series, volume_series)
```

30.3 Fear & Greed Index

Composite proxy score (0-100) based on four components: volatility (vs 90-day avg), momentum (vs 90-day avg), volume (vs 90-day avg), and drawdown from all-time high. Lower scores indicate fear (buying opportunity), higher scores indicate greed (caution).

```
CryptoOnChainAnalyzer.fear_greed_proxy(price_series, volume_series)
```

Fear & Greed Components

Component	Weight	Calculation
Volatility	25%	Current 21d vol vs 90d avg vol
Momentum	25%	Current return vs 90d avg return
Volume	25%	Current volume vs 90d avg volume
ATH Drawdown	25%	Distance from all-time high

Score interpretation: 0-25 = Extreme Fear, 25-50 = Fear, 50-75 = Greed, 75-100 = Extreme Greed. Extreme fear may signal buying opportunities; extreme greed may signal tops.

Supported Cryptocurrencies

BTC (Bitcoin), ETH (Ethereum), SOL (Solana), ADA (Cardano), DOT (Polkadot), LINK (Chainlink), AVAX (Avalanche), MATIC (Polygon). Data sourced from CoinGecko free API using `urllib.request` with no additional dependencies.

31. Insider Trading Tracker

The Insider Trading Tracker monitors SEC Form 4 filings to surface significant insider transactions. Research shows that aggregate insider buying (especially cluster buys by multiple insiders) has predictive power for future stock returns.

31.1 Insider Sentiment Score

The InsiderTracker computes a sentiment score (-100 to +100) from SEC Form 4 filings. C-suite transactions (CEO, CFO, COO) receive higher weights than other insiders. Buy transactions are positive, sells negative.

```
InsiderTracker.insider_sentiment_score(transactions_df, lookback_days=90)
```

31.2 Cluster Detection

Flags coordinated insider activity: 3+ insiders trading in the same direction within a 30-day window. Cluster buys are historically predictive of positive forward returns.

```
InsiderTracker.cluster_buy_detection(transactions_df, window_days=30,  
min_insiders=3)
```

31.3 Forward Return Analysis

Compares forward returns (30d, 60d, 90d) after insider buy vs sell transactions, overlaid on the price chart with buy/sell markers. Provides empirical evidence on insider trading signal efficacy.

```
InsiderTracker.insider_vs_price(transactions_df, prices_df)  
InsiderTracker.top_insider_trades(transactions_df, n=20)
```

Sentiment Score Calculation

The insider sentiment score is computed as a weighted net buy ratio:

$$\text{Score} = 100 * (\text{WeightedBuys} - \text{WeightedSells}) / (\text{WeightedBuys} + \text{WeightedSells})$$

C-suite transactions (CEO, CFO, COO, President) receive 3x weight; Directors receive 2x weight; other insiders receive 1x weight. The score ranges from -100 (all insider selling) to +100 (all insider buying). Values above +30 are considered bullish; below -30 are bearish.

31.3 Signal Classification

Signal Type	Definition	Historical Edge	Confidence
Cluster Buy	3+ insiders buying within 30 days	+8.2% 6-month avg excess return	High
CEO/CFO Purchase	C-suite open market purchase	+5.4% 6-month avg excess return	High
Large Purchase	Transaction > \$500K	+4.1% 6-month avg excess return	Medium

Signal Type	Definition	Historical Edge	Confidence
Director Buy	Board member purchase	+2.8% 6-month avg excess return	Medium
10% Owner Sale	Major shareholder selling	-1.2% 6-month avg excess return	Low
Planned Sale (10b5-1)	Pre-scheduled disposition	Not informative	N/A

31.4 Data Source and Parsing

Insider transaction data is sourced from SEC EDGAR Form 4 XML filings. The parser extracts: reporting person name, title, transaction type (P=Purchase, S=Sale, A=Award), shares traded, price, and ownership type (Direct/Indirect). Transactions are classified as informative (open market) or non-informative (option exercises, gifts, 10b5-1 plans).

```
class InsiderTradingTracker:
    def fetch_form4_filings(self, ticker: str, lookback_days: int = 90) -> pd.DataFrame
    def classify_transaction(self, filing: dict) -> str # informative / non-informative
    def detect_cluster_buys(self, transactions: pd.DataFrame, window: int = 30) -> list
    def compute_insider_sentiment(self, transactions: pd.DataFrame) -> float # -1 to +1
    def plot_insider_activity(self, ticker: str) -> plotly.Figure
```

31.5 Insider Sentiment Score

An aggregate insider sentiment score is computed for each ticker:

$$\text{Insider Sentiment} = (\text{Buy Volume} - \text{Sale Volume}) / (\text{Buy Volume} + \text{Sale Volume})$$

The score ranges from -1 (all selling) to +1 (all buying). Only informative transactions are included. The score is computed over a rolling 90-day window and normalized against the stock's historical insider activity to account for firm-specific patterns.

32. Watchlist & Alerts

32.1 Watchlist Management

The WatchlistManager enables creating and managing named watchlists stored in session state. Each watchlist snapshot shows current price, change%, volume, RSI, SMA 50/200, and 52-week range.

```
WatchlistManager.create_watchlist(name, tickers)
WatchlistManager.get_watchlist_snapshot(name)
```

32.2 Alert Engine

10 configurable alert types: price above/below threshold, SMA crossover (up/down), RSI overbought/oversold, volume spike, drawdown, earnings proximity, and percent change. Alerts are checked against live data and trigger notifications.

```
WatchlistManager.add_alert(ticker, alert_type, params)
WatchlistManager.check_alerts(prices_cache)
```

32.3 Performance Heatmap

Heatmap visualization of watchlist performance across multiple timeframes (1D, 1W, 1M, 3M, 6M, 1Y). Color-coded returns enable quick identification of momentum and laggards.

```
WatchlistManager.watchlist_heatmap_data(name)
```

Alert Types

Alert Type	Parameters	Trigger Condition
Price Above	threshold	Current price > threshold
Price Below	threshold	Current price < threshold
SMA Cross Up	sma_period	Price crosses above SMA
SMA Cross Down	sma_period	Price crosses below SMA
RSI Overbought	level (default 70)	RSI > level
RSI Oversold	level (default 30)	RSI < level
Volume Spike	multiplier (default 2x)	Volume > multiplier * 20d avg
Drawdown	threshold	Drawdown from peak > threshold
Earnings	days_ahead	Earnings within N days
Percent Change	threshold, period	Change > threshold over period

32.4 Alert Condition Types

Alert Type	Condition	Example	Check Frequency
Price Threshold	Price crosses user-defined level	AAPL > \$200	Every 5 min
Percent Change	Daily/weekly change exceeds threshold	TSLA down > 5% intraday	Every 15 min
Volume Spike	Volume exceeds N * avg volume	NVDA volume > 3x 20-day avg	Every 15 min
Technical Signal	Indicator triggers	RSI(14) < 30 (oversold)	Daily
Earnings Upcoming	Earnings within N days	MSFT reports in 3 days	Daily
Correlation Break	Rolling corr drops below threshold	AAPL-MSFT corr < 0.5	Daily
Sentiment Shift	Sentiment score change > threshold	Sentiment flips negative	Hourly

32.5 Watchlist Management

Watchlists are stored in Streamlit session state and persisted to local JSON files. Each watchlist entry contains: ticker symbol, display name, date added, custom notes, and active alert configurations. Users can create multiple named watchlists (e.g., 'Tech Picks', 'Income Portfolio', 'Watchlist Q2') and switch between them.

```
class WatchlistManager:
    def add_ticker(self, watchlist: str, ticker: str, notes: str = "") -> None
    def remove_ticker(self, watchlist: str, ticker: str) -> None
    def set_alert(self, ticker: str, alert_type: str, condition: dict) -> None
    def check_alerts(self) -> list[dict] # triggered alerts
    def export_watchlist(self, watchlist: str, format: str = "csv") -> bytes
    def import_watchlist(self, data: bytes, format: str = "csv") -> None
```

The alert engine runs as a background check on each page load (not a continuous polling loop). For more frequent monitoring, users can enable auto-refresh in Settings with a configurable interval (minimum 60 seconds).

33. ML-Powered Top 10 Assets

The MLAssetRanker scans approximately 80 liquid assets across stocks (30), ETFs (18), crypto (10), forex (8), and commodities (10) to identify the top investment opportunities using an ensemble of three ML models.

33.1 Feature Engineering

20 features are computed per asset spanning momentum, mean-reversion, volatility, technical, volume, and risk categories. Features include multi-horizon returns, RSI, SMA ratios, Bollinger %B, volume trends, Sharpe, max drawdown, skewness, and kurtosis.

```
MLAssetRanker._compute_features_for_series(close, volume) # 20 features
```

33.2 Ensemble Scoring

Three models are ensembled with fixed weights:

$$\text{Score} = 0.40 * \text{composite_z} + 0.30 * \text{rf_score} + 0.30 * \text{cluster_score}$$

Where `composite_z` is a multi-factor z-score ranking, `rf_score` is a Random Forest attractiveness prediction trained on historical features, and `cluster_score` uses K-Means cluster quality via PCA.

```
MLAssetRanker.composite_score(features_df) # z-score ranking
MLAssetRanker.rf_score(features_df, historical_features_df) # RF prediction
MLAssetRanker.cluster_score(features_df, historical_features_df) # K-Means quality
```

33.3 Asset Explanations

Each top pick includes a natural-language explanation of why it was selected, highlighting its strongest features (e.g., high momentum, low drawdown, favorable mean-reversion setup).

```
MLAssetRanker.explain_pick(ticker)
```

33.4 Model Diagnostics

Feature importance from the Random Forest model, PCA cluster scatter plot, score distribution histogram, and a full-universe treemap color-coded by ensemble score. Category breakdown donut chart shows the distribution of top picks across asset classes.

```
MLAssetRanker.rank_assets(prices_dict)
MLAssetRanker.get_top_n(n=10)
MLAssetRanker.category_breakdown()
```

Universe Coverage

Category	Count	Examples
Stocks	30	AAPL, MSFT, NVDA, TSLA, AMZN, GOOGL, META, JPM, V
ETFs	18	SPY, QQQ, IWM, EEM, GLD, TLT, XLK, XLF, XLE
Crypto	10	BTC-USD, ETH-USD, SOL-USD, ADA-USD, DOT-USD
Forex	8	EURUSD=X, GBPUSD=X, USDJPY=X, AUDUSD=X

Category	Count	Examples
Commodities	10	GC=F, SI=F, CL=F, NG=F, ZC=F, ZW=F

20-Feature Vector

#	Feature	Category
1-4	Return 1d, 5d, 21d, 63d	Momentum
5-6	Return 126d, 252d	Long-term momentum
7-8	SMA50 ratio, SMA200 ratio	Mean-reversion
9	Bollinger %B	Mean-reversion
10-11	Volatility 21d, 63d	Volatility
12	ATR (Average True Range)	Volatility
13-14	RSI 14, MACD histogram	Technical
15-16	Volume ratio, Volume trend	Volume
17-20	Sharpe, Max DD, Skewness, Kurtosis	Risk

34. News Sentiment Analyzer

34.1 TF-IDF + Naive Bayes

The NewsSentimentAnalyzer uses a TF-IDF + ComplementNB classifier trained semi-supervised on lexicon-labeled headlines. A 40-word financial sentiment lexicon provides initial labels; the NB classifier learns to generalize from these.

$$\text{TF-IDF: } \text{tf}(t,d) * \log(N / \text{df}(t))$$
$$\text{Ensemble Score} = 0.40 * \text{lexicon_score} + 0.60 * \text{nb_score}$$

```
NewsSentimentAnalyzer.train_nb_classifier(headlines_with_labels)
NewsSentimentAnalyzer.score_headlines(headlines)
```

34.2 Divergence Detection

Identifies cases where sentiment and price disagree over a lookback window: positive news + falling price (potential bullish divergence) or negative news + rising price (potential bearish divergence). These divergences can signal upcoming reversals.

```
NewsSentimentAnalyzer.detect_divergences(sentiment_ts, price_series, lookback=10)
```

34.3 Sentiment-Price Correlation

Analyzes the relationship between sentiment and price at multiple lags: concurrent (same day), predictive (sentiment leads price by 1d and 5d), and lagged (price leads sentiment). A high predictive correlation suggests sentiment has forecasting power.

```
NewsSentimentAnalyzer.sentiment_price_correlation(sentiment_ts, price_series)
```

Correlation Analysis Detail

Lag Type	Calculation	Interpretation
Concurrent	<code>corr(sentiment_t, return_t)</code>	Same-day relationship
Predictive 1d	<code>corr(sentiment_t, return_{t+1})</code>	Sentiment leads price by 1 day
Predictive 5d	<code>corr(sentiment_t, return_{t+5})</code>	Sentiment leads price by 5 days
Lagged 1d	<code>corr(sentiment_t, return_{t-1})</code>	Price leads sentiment by 1 day

A statistically significant predictive correlation ($p < 0.05$) suggests that sentiment analysis has genuine forecasting power for the given asset. The word frequency chart from recent headlines helps identify dominant themes and narratives.

34.4 News Impact Scoring

Each news article receives an impact score based on multiple factors:

Factor	Weight	Calculation
Sentiment Magnitude	0.30	Absolute compound VADER score
Source Authority	0.25	Pre-assigned weight by source (Reuters=1.0, blog=0.3)

Factor	Weight	Calculation
Recency	0.20	Exponential decay with 24-hour half-life
Relevance	0.15	TF-IDF cosine similarity to ticker keywords
Uniqueness	0.10	1 - max similarity to other recent articles (dedup)

$$\text{Impact}_i = \sum_k w_k * \text{Factor}_{k,i}$$

34.5 Temporal Aggregation

News sentiment is aggregated at multiple time scales to capture both breaking news impact and sustained narrative shifts:

- Real-time (5-min bins): Detects sudden sentiment spikes from breaking news
- Hourly rolling: Smoothed signal for intraday sentiment trends
- Daily aggregate: Impact-weighted mean across all articles for the day
- Weekly trend: Detects sustained positive/negative narrative shifts

The aggregated sentiment time series is aligned with price data for correlation analysis. A Granger causality test is applied to determine if sentiment changes lead price changes (and vice versa) for each ticker.

34.6 Implementation

```
class NewsSentimentAnalyzer:
    def fetch_news(self, ticker: str, days_back: int = 7) -> list[dict]
    def score_article(self, article: dict) -> dict # {sentiment, impact, relevance}
    def aggregate_sentiment(self, articles: list, freq: str = "1D") -> pd.Series
    def granger_test(self, sentiment: pd.Series, returns: pd.Series, lags: int = 5) -> dict
    def plot_sentiment_overlay(self, ticker: str) -> plotly.Figure
    def generate_summary(self, articles: list, top_n: int = 5) -> str
```


35. Smart Portfolio Constructor

35.1 Ledoit-Wolf Covariance

The `SmartPortfolioConstructor` uses Ledoit-Wolf shrinkage for robust covariance estimation, addressing the estimation error problem in mean-variance optimization with high-dimensional data.

$$\text{Sigma_shrunk} = \alpha * T + (1 - \alpha) * S$$

Where T is the structured target (scaled identity), S is the sample covariance, and α is the optimal shrinkage intensity estimated by minimizing expected loss.

```
SmartPortfolioConstructor.ledoit_wolf_covariance(returns)
```

35.2 Black-Litterman Framework

Market-implied equilibrium returns are computed using reverse optimization from market-cap weights. These are combined with investor views via Bayesian updating.

$$P_i = \text{delta} * \text{Sigma} * w_{\text{mkt}} \text{ (equilibrium returns)}$$

$$\mu_{\text{BL}} = [(\tau * \text{Sigma})^{-1} + P' * \Omega^{-1} * P]^{-1} * [(\tau * \text{Sigma})^{-1} * P_i + P' * \Omega^{-1} * Q]$$

```
SmartPortfolioConstructor.market_implied_returns(market_cap_weights, cov_matrix)
SmartPortfolioConstructor.black_litterman_returns(
    implied_returns, P, Q, Omega, cov_matrix)
```

35.3 ML-Derived Views

Momentum-based signals are converted into Black-Litterman views. Assets with strong positive momentum are assigned positive views, and vice versa. View confidence is proportional to signal strength.

```
SmartPortfolioConstructor.ml_views(features_df, rf_model)
```

35.4 Regime Monte Carlo

Portfolio weights are stress-tested via regime-conditional Monte Carlo simulation. Return distributions are sampled from the historical return distribution, conditioned on the current macro regime. Output: fan chart with percentile bands (5th, 25th, 50th, 75th, 95th).

```
SmartPortfolioConstructor.regime_conditional_monte_carlo(
    weights, returns, n_sims, horizon)
```

Strategy comparison table: Smart Portfolio vs Equal-Weight vs Min-Variance vs Max-Sharpe across return, volatility, Sharpe, max drawdown, and CVaR.

Optimization Parameters

The portfolio optimization step uses `scipy.optimize.minimize` with the SLSQP solver. Risk tolerance maps to the optimization objective:

Risk Tolerance	Objective	Max Weight
Conservative	Minimize vol, return $\geq R_f + 2\%$	0.15
Moderate	Maximize Sharpe ratio	0.25
Aggressive	Maximize return, vol $\leq 25\%$	0.50

Regime Monte Carlo Details

The regime-conditional Monte Carlo uses a 2-state Hidden Markov Model fitted to portfolio returns. In the 'normal' regime, returns are drawn from $N(\mu_{\text{normal}}, \sigma_{\text{normal}})$. In the 'stress' regime, returns are drawn from $N(\mu_{\text{stress}}, \sigma_{\text{stress}})$ where $\mu_{\text{stress}} < 0$ and $\sigma_{\text{stress}} > \sigma_{\text{normal}}$. The current regime determines the initial state for forward simulation.

Configuration

- Risk Tolerance: `st.session_state['smart_risk_tol']` (conservative/moderate/aggressive)
- Max Weight per Asset: `st.session_state['smart_max_weight']` (0.05-0.50, default 0.25)
- Monte Carlo Sims: `st.session_state['smart_mc_sims']` (500-5000, default 1000)

36. Risk Decomposition Engine

36.1 Factor Regression

The RiskDecompositionEngine performs OLS factor regression per asset using six factor proxy ETFs: SPY (market), IWM (size), IWD (value), MTUM (momentum), QUAL (quality), USMV (low vol).

$$r_i = \alpha + \beta_{\text{mkt}} * r_{\text{SPY}} + \beta_{\text{size}} * r_{\text{IWM}} + \beta_{\text{val}} * r_{\text{IWD}} + \dots + \epsilon$$

Returns: alpha, betas, R-squared, t-statistics, and residual volatility per asset.

```
RiskDecompositionEngine.factor_regression(asset_returns, factor_returns)
```

36.2 Return & Risk Attribution

Return attribution decomposes portfolio return into factor contributions and alpha via a waterfall chart. Risk attribution splits portfolio variance into factor risk (systematic) and idiosyncratic risk using factor betas and the factor covariance matrix.

```
RiskDecompositionEngine.return_attribution(portfolio_return, factor_returns_row,
factor_betas)
RiskDecompositionEngine.risk_attribution(weights, cov_matrix, factor_betas,
factor_cov)
```

36.3 Style Box Classification

Morningstar-style 3x3 grid classification based on market beta (market sensitivity), size beta (small vs large cap), and value beta (value vs growth). Rolling factor exposure tracking with drift alerts when exposures move more than 2 standard deviations from their mean.

```
RiskDecompositionEngine.style_box(market_beta, size_beta, value_beta)
RiskDecompositionEngine.factor_drift_detection(rolling_betas, lookback=63)
```

Factor Proxy ETFs

Factor	ETF Proxy	Description
Market	SPY	S&P 500 total market return
Size	IWM	Russell 2000 small-cap return
Value	IWD	Russell 1000 Value return
Momentum	MTUM	MSCI USA Momentum Factor
Quality	QUAL	MSCI USA Quality Factor
Low Volatility	USMV	MSCI USA Min Vol Factor

Style Box Grid

The 3x3 style box maps (market_beta, size_beta) to a grid: Large/Mid/Small (rows) x Value/Blend/Growth (columns). Classification thresholds: market_beta > 1.1 = aggressive, < 0.9 = defensive; value_beta > 0.3 = value, < -0.3 = growth; size_beta > 0.3 = small-cap, < -0.3 =

large-cap.

37. ML Price Forecaster

The ML Price Forecaster provides probabilistic price predictions using an ensemble of machine learning models. It generates point forecasts with confidence intervals for user-specified horizons (1-day, 5-day, 20-day, 60-day).

37.1 Rolling Regression

The MLPriceForecaster uses a rolling linear regression with an expanding training window as the interpretable baseline. Weight: 25% in the ensemble.

```
MLPriceForecaster.rolling_regression_forecast(X, y, train_window=252,
forecast_horizon=21)
```

37.2 Gradient Boosting

GradientBoostingRegressor with walk-forward validation. The strongest model in the ensemble (45% weight). Feature importance is extracted for explainability.

```
MLPriceForecaster.gradient_boosting_forecast(X, y, train_window=252,
forecast_horizon=21)
```

37.3 ARIMA + EWMA Ensemble

ARIMA models the mean return process; EWMA (Exponentially Weighted Moving Average) forecasts volatility to generate confidence intervals. Combined weight: 30%.

$$\text{ARIMA}(p,d,q): r_t = c + \text{Sum}(\phi_i * r_{t-i}) + \text{Sum}(\theta_j * \text{epsilon}_{t-j}) + \text{epsilon}_t$$

$$\text{EWMA vol: } \sigma^2_t = \lambda * \sigma^2_{t-1} + (1-\lambda) * r^2_{t-1}$$

$$\text{Ensemble} = 0.25 * \text{LR} + 0.45 * \text{GBR} + 0.30 * \text{ARIMA}$$

Model disagreement metric with High/Medium/Low confidence labels based on the coefficient of variation across model forecasts. Backtest accuracy panel: MAE, RMSE, directional accuracy.

```
MLPriceForecaster.ensemble_forecast(forecast_horizon=21)
MLPriceForecaster.backtest_accuracy(n_backtests=10)
```

37.3 Feature Engineering Pipeline

Feature Category	Features	Count	Lookback
Price Momentum	Returns (1d,5d,20d,60d), ROC, MACD signal	8	1–60 days
Volatility	Realized vol, Parkinson, Garman-Klass, ATR	6	10–60 days
Volume	Volume ratio, OBV slope, VWAP deviation	5	5–20 days
Technical	RSI, Stochastic, Bollinger %B, ADX	8	14–50 days

Feature Category	Features	Count	Lookback
Regime	HMM state, trend strength, support/resistance	4	20–100 days
Calendar	Day of week, month, quarter-end flag, OpEx	5	N/A
Cross-Asset	VIX level, DXY change, yield curve slope	4	1–20 days

Total feature vector dimension: ~40 features per observation. Features are standardized using a rolling z-score (60-day window) to ensure stationarity and prevent lookahead bias.

37.4 Model Ensemble Architecture

The forecaster combines three model types using a stacking approach:

- Gradient Boosted Trees (XGBoost): Captures non-linear feature interactions. 500 trees, max_depth=6, learning_rate=0.05.
- Ridge Regression: Linear baseline that excels in low-signal-to-noise regimes. Regularization alpha selected via cross-validation.
- LSTM Neural Network: Sequence model for temporal dependencies. 2 layers, 64 hidden units, dropout=0.2, trained for 50 epochs.

$$\hat{y}_{\text{ensemble}} = w_1 \hat{y}_{\text{XGB}} + w_2 \hat{y}_{\text{Ridge}} + w_3 \hat{y}_{\text{LSTM}}$$

Ensemble weights are determined by inverse validation MSE, updated weekly during the walk-forward process. The meta-learner adds a bias correction term based on recent forecast errors.

37.5 Confidence Intervals

Prediction intervals are generated using conformal prediction. The method is distribution-free and provides finite-sample coverage guarantees:

$$CI_{\alpha} = [\hat{y} - q_{1-\alpha/2}(|\text{residuals}|), \hat{y} + q_{1-\alpha/2}(|\text{residuals}|)]$$

Default confidence levels are 80% and 95%. The residuals are computed from a calibration set (most recent 20% of the training window) to ensure the intervals are well-calibrated for current market conditions.

37.6 Implementation

```
class MLPriceForecaster:
    def build_features(self, prices: pd.DataFrame) -> pd.DataFrame
    def train_ensemble(self, X: pd.DataFrame, y: pd.Series) -> dict # model weights
    def predict(self, X_new: pd.DataFrame) -> dict # {point, ci_80, ci_95}
    def walk_forward_evaluate(self, prices: pd.DataFrame, horizon: int) -> dict
    def plot_forecast(self, ticker: str, horizon: int = 20) -> plotly.Figure
```

38. Earnings Surprise Predictor

38.1 Pre-Earnings Features

The EarningsSurprisePredictor extracts pre-earnings features in a window before each earnings date: momentum (cumulative return), volatility, RSI, volume trend, historical beat rate, and number of consecutive beats.

```
EarningsSurprisePredictor.compute_pre_earnings_features(prices, earnings_dates,
window_pre=30)
```

38.2 RandomForest Classifier

A RandomForestClassifier is trained with TimeSeriesSplit cross-validation to predict beat vs miss. Output: beat probability (0-100%) with Likely Beat / Likely Miss / Toss-Up labels.

```
EarningsSurprisePredictor.train_predictor(features_df)
EarningsSurprisePredictor.predict_next_earnings(current_features)
```

38.3 Walk-Forward Validation

Historical prediction performance is evaluated via walk-forward validation: train on all data up to each earnings date, predict the next event, and accumulate accuracy statistics. Feature importance analysis identifies which pre-earnings signals are most predictive.

```
EarningsSurprisePredictor.historical_prediction_performance(features_df)
EarningsSurprisePredictor.earnings_calendar_upcoming(tickers, days_ahead=30)
```

38.4 Feature Importance Analysis

Feature	Importance (Gini)	Description
Historical Beat Rate	0.24	Fraction of past earnings that beat consensus
Pre-Earnings Momentum	0.19	Cumulative return in 30-day pre-announcement window
Consecutive Beats	0.15	Number of consecutive quarters beating estimates
Pre-Earnings Volatility	0.13	Realized volatility in pre-announcement window
RSI (14-day)	0.11	Relative Strength Index entering earnings
Volume Trend	0.10	Slope of 20-day volume regression
Sector Momentum	0.08	Sector ETF performance relative to SPY

38.5 Post-Earnings Drift Analysis

The module also analyzes Post-Earnings Announcement Drift (PEAD), the well-documented anomaly where stocks continue to drift in the direction of an earnings surprise for 60-90 days. The drift magnitude is computed as:

$$PEAD_t = CAR_{t+1:t+60} = \sum (R_{stock,i} - R_{market,i})$$

where CAR is the Cumulative Abnormal Return over the 60 trading days following the announcement. The predictor outputs both the beat/miss probability and the expected drift magnitude based on the historical PEAD for similar surprise magnitudes.

38.6 Configuration Parameters

Parameter	Default	Range	Description
window_pre	30	10–90	Days before earnings to compute features
n_estimators	200	50–500	Number of trees in RandomForest
cv_splits	5	3–10	TimeSeriesSplit folds for cross-validation
min_samples_leaf	5	2–20	Minimum samples per leaf node
beat_threshold	0.55	0.50–0.70	Probability cutoff for Likely Beat label
days_ahead	30	7–90	Look-ahead for upcoming earnings calendar

A. Appendix: Supported Asset Classes

Asset Class	Example Tickers	Notes
Stocks & ETFs	NVDA, TSLA, AAPL, MSFT, GOOGL	Full support: fundamentals, options, technicals
Crypto	BTC-USD, ETH-USD, SOL-USD	Price data via yfinance; on-chain via CoinGecko
Forex	EURUSD=X, GBPUSD=X, USDJPY=X	Price data only; no fundamentals
Commodities	GC=F (Gold), CL=F (Oil), SI=F (Silver)	Futures contracts; no fundamentals
Options	AAPL, MSFT, NVDA, TSLA, AMZN	Live chains from Yahoo Finance

22 one-click presets are organized by category: Stocks (11 presets), Crypto (2), ETFs (4), Forex (2), and Commodities (3). Each preset selects a curated basket of 4-8 tickers.

A.1 Stock Presets

Preset Name	Tickers	Description
FAANG+	AAPL, MSFT, GOOGL, AMZN, META, NVDA, TSLA, NFLX	Mega-cap tech leaders
Dividend Aristocrats	JNJ, KO, PG, MMM, ABT, WMT, PEP, MCD	25+ years consecutive dividend increases
Semiconductor	NVDA, AMD, INTC, TSM, AVGO, QCOM, TXN, MU	Chip makers and foundries
Banking	JPM, BAC, GS, MS, WFC, C	Major US banks
Healthcare	UNH, JNJ, PFE, ABBV, TMO, MRK, LLY, ABT	Pharma, biotech, and services
Energy	XOM, CVX, COP, SLB, EOG, MPC	Oil majors and services
Small Cap Growth	Selected via market-cap filter	Russell 2000 high-growth subset

A.2 ETF, Crypto, and Commodity Presets

Category	Preset	Tickers
ETFs	Sector SPDRs	XLK, XLF, XLE, XLV, XLI, XLP, XLU, XLRE, XLC, XLY, XLB
ETFs	Factor ETFs	MTUM, QUAL, VLUE, SIZE, USMV
ETFs	Bond ETFs	AGG, TLT, HYG, LQD, TIP, SHY
ETFs	International	EFA, EEM, VWO, IEFA
Crypto	Major Coins	BTC-USD, ETH-USD, SOL-USD, ADA-USD
Crypto	DeFi Tokens	UNI-USD, AAVE-USD, LINK-USD, MKR-USD
Commodities	Precious Metals	GC=F, SI=F, PL=F
Commodities	Energy	CL=F, NG=F, BZ=F
Commodities	Agriculture	ZC=F, ZS=F, ZW=F

Category	Preset	Tickers
Forex	Major Pairs	EURUSD=X, GBPUSD=X, USDJPY=X, USDCHF=X
Forex	EM Pairs	USDBRL=X, USDMXN=X, USDINR=X

B. Appendix: Dependencies

QuantLab requires Python 3.9+ and the following library ecosystem. All dependencies are installable via pip. The application is tested against the pinned versions shown below.

Package	Purpose	Version
streamlit	Web application framework	>=1.56
yfinance	Market data fetching	latest
pandas	Data manipulation	latest
numpy	Numerical computing	latest
plotly	Interactive visualization	latest
scipy	Optimization & statistics	latest
statsmodels	Time series & econometrics	latest
scikit-learn	Machine learning models	latest
ta	Technical indicators	latest
xlsxwriter	Excel export	latest
matplotlib	Chart rendering for exports	latest
seaborn	Statistical visualization	latest
fpdf2	PDF/slide generation	latest
tzdata	Timezone data	latest

Install all dependencies: `pip install -r requirements.txt`

B.2 Complete Dependency Matrix

Package	Version	Purpose	License
streamlit	≥ 1.28	Web application framework	Apache 2.0
pandas	≥ 2.0	Data manipulation and analysis	BSD 3-Clause
numpy	≥ 1.24	Numerical computing	BSD 3-Clause
plotly	≥ 5.15	Interactive charting	MIT
yfinance	≥ 0.2.28	Yahoo Finance data	Apache 2.0
scipy	≥ 1.11	Scientific computing, optimization	BSD 3-Clause
scikit-learn	≥ 1.3	Machine learning algorithms	BSD 3-Clause
statsmodels	≥ 0.14	Statistical models, cointegration	BSD 3-Clause
textblob	≥ 0.17	NLP sentiment analysis	MIT
vaderSentiment	≥ 3.3	Social media sentiment	MIT

Package	Version	Purpose	License
fredapi	≥ 0.5	FRED economic data	Apache 2.0
requests	≥ 2.31	HTTP client	Apache 2.0
hmmlearn	≥ 0.3	Hidden Markov Models	BSD 3-Clause
arch	≥ 6.0	GARCH volatility models	NCSA
copulas	≥ 0.9	Copula dependence modeling	MIT
ta	≥ 0.10	Technical analysis indicators	MIT
openpyxl	≥ 3.1	Excel export	MIT

B.3 Optional Dependencies

Package	Version	Purpose	Required For
xgboost	≥ 2.0	Gradient boosted trees	ML Price Forecaster (Sec. 37)
tensorflow	≥ 2.14	LSTM neural networks	ML Price Forecaster (Sec. 37)
pycoingecko	≥ 3.1	CoinGecko on-chain data	Crypto On-Chain (Sec. 30)
newsapi-python	≥ 0.2.7	NewsAPI client	News Sentiment (Sec. 34)
alpha_vantage	≥ 2.3	Alpha Vantage client	Multi-Source fallback (Sec. 15)

B.4 System Requirements

Requirement	Minimum	Recommended
Python	3.9	3.11+
RAM	4 GB	8 GB+
Storage	500 MB	2 GB (for cached data)
CPU	2 cores	4+ cores (for ML training)
Network	Broadband	Low-latency (for real-time data)
OS	Linux, macOS, Windows	Linux or macOS preferred

C. Appendix: Data Source Configuration

QuantLab integrates with multiple external data APIs. This appendix details the configuration requirements, API key setup, and rate limits for each provider.

API keys are entered in the sidebar under Data Sources > Configure Data Providers. Keys are stored in session state only (never written to disk). Each provider can be individually enabled/disabled.

Provider	Data	API Key	Free Tier
Yahoo Finance (yfinance)	Prices, fundamentals, quotes	Not needed	Unlimited
Alpha Vantage	Prices, fundamentals	Required	25 calls/day
FRED	Macro data (yields, CPI, GDP)	Required	Unlimited
Finnhub	Real-time quotes, company news	Required	60 calls/min

Cascading Fallback Architecture

```
DataSourceOrchestrator.fetch_prices(tickers, period)
  Tier 1: yfinance (DEFAULT)
  Tier 2: Alpha Vantage (opt-in)
  Tier 3: Finnhub (opt-in)
  Tier 4: Session cache
  Tier 5: Error with diagnostics
```

C.2 API Configuration Reference

Provider	Env Variable	Free Tier	Rate Limit	Auth Type
Yahoo Finance	None (unauthenticated)	Unlimited*	~2000 req/hr	None
Alpha Vantage	ALPHA_VANTAGE_API_KEY	500 req/day	5 req/min	API Key
FRED	FRED_API_KEY	Unlimited	120 req/min	API Key
CoinGecko	COINGECKO_API_KEY	10,000 req/month	10 req/min	API Key
NewsAPI	NEWS_API_KEY	100 req/day	1 req/sec	API Key

* Yahoo Finance is accessed via the yfinance library which reverse-engineers the Yahoo Finance API. While there is no official rate limit, aggressive usage may trigger temporary IP blocks. The built-in caching layer mitigates this risk.

C.3 Environment Setup

API keys are configured via environment variables or a .env file in the project root. The application loads these at startup using python-dotenv:

```
# .env file (project root) ALPHA_VANTAGE_API_KEY=your_key_here
FRED_API_KEY=your_key_here COINGECKO_API_KEY=your_key_here
NEWS_API_KEY=your_key_here # Or export directly: export
ALPHA_VANTAGE_API_KEY=your_key_here
```

C.4 Data Freshness and Caching

Data Type	Default TTL	Staleness Alert	Force Refresh
Intraday Prices	15 minutes	After 1 hour	st.cache_data.clear()
Daily OHLCV	End of trading day	After 24 hours	Manual refresh button
Fundamentals	7 days	After 14 days	Manual refresh button
Options Chains	5 minutes	After 30 minutes	Auto on each view
News Articles	15 minutes	After 1 hour	Auto on each view
Economic Indicators	24 hours	After 48 hours	Manual refresh button
ML Model Predictions	On demand	Never (computed fresh)	Re-train button

C.5 Troubleshooting Common Data Issues

Issue	Likely Cause	Solution
Empty price data	Ticker delisted or invalid	Verify ticker symbol on Yahoo Finance
Stale quotes	Cache not refreshed	Click refresh or clear st.cache_data
Rate limit errors (429)	Too many API calls	Wait 60 seconds; check rate limit config
Missing fundamentals	Not available for ETFs/crypto	Only stocks have full fundamentals
Options chain empty	No options for this ticker	Options only available for US-listed equities
Timeout errors	Slow network or provider outage	Check network; try alternative provider

D. Appendix: ML Model Specifications

Model	Module	Algorithm	Key Parameters
Price Prediction (3x)	Section 10	LR / RF / GBR	n_estimators=100-200, max_depth=10
Asset Clustering	Section 11	K-Means / GMM	n_clusters=3, n_init=10
Anomaly Detection	Section 11	Isolation Forest	contamination=0.01-0.15
Regime Detection	Section 11	GMM	n_components=3, full covariance
Sentiment (NB)	Section 34	ComplementNB + TF-IDF	alpha=1.0, max_features=5000
Asset Ranking (RF)	Section 33	RandomForest Classifier	n_estimators=100, 20 features
Asset Ranking (KM)	Section 33	K-Means + PCA	n_clusters=5, n_components=2
Price Forecast (GBR)	Section 37	GradientBoosting	n_estimators=200, lr=0.1
Price Forecast (ARIMA)	Section 37	ARIMA(p,d,q) + EWMA	order auto-selected
Earnings Predictor	Section 38	RandomForest Classifier	TimeSeriesSplit CV
Portfolio (LW)	Section 35	Ledoit-Wolf Shrinkage	alpha auto-estimated

Feature Engineering Summary

ML models across QuantLab use a consistent set of feature categories:

- Momentum: returns at 1d, 5d, 21d, 63d, 126d, 252d horizons
- Mean-Reversion: SMA ratios (P/SMA20, P/SMA50, P/SMA200), Bollinger %B
- Volatility: rolling standard deviation (21d, 63d), EWMA vol, ATR
- Technical: RSI, MACD histogram, Bollinger bandwidth
- Volume: volume ratio (current / 20d avg), volume trend slope
- Risk: Sharpe ratio, max drawdown, skewness, kurtosis

Cross-Validation Strategy

All ML models use strictly chronological train/test splits with no lookahead. TimeSeriesSplit is used for cross-validation. Walk-forward evaluation is the standard: train on expanding window, predict the next period, advance one step, and repeat.

Ensemble Architecture

QuantLab employs multiple ensemble patterns across its ML modules:

Module	Ensemble Type	Weights	Combination
ML Predictions (S10)	Independent	Equal	Side-by-side comparison
Top 10 Assets (S33)	Weighted avg	40/30/30	Composite score
News Sentiment (S34)	Weighted avg	40/60	Lexicon + NB blend
Price Forecast (S37)	Weighted avg	25/45/30	LR + GBR + ARIMA

Module	Ensemble Type	Weights	Combination
Earnings Predictor (S38)	Single model	100	RF with CV
Clustering (S11)	Alternative	N/A	User selects method

Hyperparameter Ranges

Parameter	Default	Range	Module
n_estimators (RF)	100	50-500	ML Predictions, Top 10
max_depth (RF)	10	5-20	ML Predictions
n_estimators (GBR)	200	100-500	Price Forecast
learning_rate (GBR)	0.1	0.01-0.3	Price Forecast
n_clusters (KMeans)	3	2-10	Clustering, Top 10
contamination (IF)	0.05	0.01-0.15	Anomaly Detection
alpha (ComplementNB)	1.0	0.1-10.0	News Sentiment
max_features (TF-IDF)	5000	1000-10000	News Sentiment
n_splits (TSCV)	5	3-10	Earnings Predictor
shrinkage (LW)	auto	0-1	Smart Portfolio